FYS-STK4155 - Project 1

Carl Petter Duedahl, Ole Morten Gården, Ingi Benedikt Jónasson
University of Oslo
(Dated: October 5, 2026)

Behind the scenes of AI algorithms, simple regression methods such as OLS and Ridge are performed, together with differentiation methods such as automatic differentiation, to find fits that can then be used to make predictions, typically achieved via gradient descent or adaptive gradient algorithms. However, these methods have their limitations, especially when the functions we study have undesirable mathematical properties which cannot easily be overcome. These problems can be minimized through proper analysis to achieve better fits without exceptional drawbacks. We study the Runge function to find out which combination of methods and parameters provides and strikes the best balance between resource efficiency and overall fit. Additionally, we study the limitations of those methods with regards to the Runge function.

All relevant code and data can be found here: `github.com/carlpd/FYS-STK/Project1`

I. INTRODUCTION

Machine learning has seen great progress in recent years, especially neural networks due to more computing power. However, for simple cases, linear regression can still be a better solution, due to its simpler and more computationally efficient nature. In this project, we tested different methods of linear regression against data simulated from a function. We used the Runge function. As explained in subsection II A, the Runge function is hard to predict for usual regression models, meaning that we could see the strengths and weaknesses of the different methods we tested. We started with the Ordinary Least Squares (OLS) method as this is a very simple model and can work as a base comparing other methods. Then we complicated a bit more by adding a penalty term λ. In the end we also introduced the Lasso method, which made our functions no longer analytically solvable.

We then tried the different resampling techniques bootstrap and k-fold cross-validation. The bootstrap draws random data points with replacements to make an estimation of the model and is useful as it does not depend on assumptions such as a Gaussian distributed error [1]. Like the OLS, it may not often be the best, but it is quite simple. k-fold cross validation shuffles the data into k groups called folds. It then trains the chosen model using all folds except one and uses the remaining one as test data. This way of evaluating the model can prove useful, especially for small datasets as you do not have to save a part of your data only for testing[1].

Next, we included methods of gradient descent including momentum, adaptive gradient algorithm [1] (AdaGrad), Root Mean Square Propagation [1] (RMSProp) and adaptive moment estimation [1] (Adam). Next we used what we had and put it into a stochastic gradient descent. Lastly, we used cross validation to find the best polynomial degree and λ for OLS, Ridge and Lasso to compare how well they worked.

This report first describes how these different methods work and their algorithms. Then it goes through how we implemented it in our code. It will then show our results and discuss what these means. In the end, it will summarize our report and think of what we have learned for future works.

II. METHODS

A. The Runge function

We use the Runge function

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1, 1]. \tag{1}$$

Although it is smooth and bounded on the real line, it has poles at $x = \pm i/5$ in the complex plane, so its Maclaurin series converges only for $|x| < 1/5$. Polynomials therefore approximate it poorly near the ends of the interval, where high-degree polynomials tend to oscillate (Runge's phenomenon). This makes it a demanding test case for polynomial regression.

B. Linear regression

1. *The regression problem*

We assume n data points generated by

$$\boldsymbol{y} = f(\boldsymbol{x}) + \boldsymbol{\varepsilon}, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{2}$$

where f is the unknown true function and the noise is independent with mean zero. We model f by a polynomial of degree $p-1$, $\tilde{y}_i = \sum_{j=0}^{p-1} \theta_j x_i^j$, or in matrix form $\tilde{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\theta}$ with $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_{p-1})^T$ and the feature matrix

$$\boldsymbol{X} = \begin{bmatrix} 1 & x_0 & \ldots & x_0^{p-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & \ldots & x_{n-1}^{p-1} \end{bmatrix} \in \mathbb{R}^{n \times p} \tag{3}$$

[1]. In our implementation the column of ones is removed and the intercept is handled by centring the data (Section II C). The task is to find the parameters that make $\tilde{\boldsymbol{y}}$ the best approximation of f, given only the noisy data. We do this with OLS, Ridge and Lasso regression.

2. Ordinary Least Squares

To find the parameters, we need a *cost function* that measures how far the model is from the data (called a measure of quality in [1]). Simply summing the residuals $y_k - \tilde{y}_k$ would not work, since positive and negative deviations would cancel. We therefore use the mean of the squared residuals,

$$C(\boldsymbol{\theta}) := \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 = \frac{1}{n}\sum_{k=0}^{n-1}(y_k - \tilde{y}_k)^2, \tag{4}$$

known as the mean squared error (MSE). Minimising (4) with respect to $\boldsymbol{\theta}$ leads to the Ordinary Least Squares (OLS) approach to linear regression.

To minimise (4), we first take the gradient:

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = -\frac{2}{n}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}). \tag{5}$$

Setting (5) to zero gives

$$\hat{\boldsymbol{\theta}}_{\text{OLS}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}. \tag{6}$$

In practice, (6) is not used directly: forming $\boldsymbol{X}^T\boldsymbol{X}$ squares the condition number of $\boldsymbol{X}$, so for high-degree polynomials the inverse is dominated by round-off. We therefore compute the solution with the pseudoinverse (`numpy.linalg.pinv`), which is based on the singular value decomposition (SVD). Since the cost is convex, any zero of the gradient is its global minimum.

3. Ridge regression

In Ridge regression, the penalty $\lambda\|\boldsymbol{\theta}\|_2^2$, with $\lambda \geq 0$, is added to the cost function[1], so that instead of minimising the MSE (4), we minimise

$$\frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2. \tag{7}$$

(Observe that setting $\lambda = 0$ yields (4).) Minimising with respect to $\boldsymbol{\theta}$ gives

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^T\boldsymbol{y}. \tag{8}$$

The factor n comes from the normalisation $1/n$ in (7). With this convention, our λ corresponds to `alpha`$= n\lambda$ in Scikit-Learn's `Ridge` [2].

The effect of the penalty is clearest in terms of the singular value decomposition $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T$, with singular values $s_1 \geq s_2 \geq \cdots \geq s_p \geq 0$ and the columns $\boldsymbol{u}_j$ of $\boldsymbol{U}$. Inserting the SVD in (8) gives the predictions on the training data as

$$\tilde{\boldsymbol{y}}_{\text{Ridge}} = \boldsymbol{X}\hat{\boldsymbol{\theta}}_{\text{Ridge}} = \sum_{j=1}^{p}\boldsymbol{u}_j\,\frac{s_j^2}{s_j^2 + n\lambda}\,\boldsymbol{u}_j^T\boldsymbol{y}. \tag{9}$$

For OLS ($\lambda = 0$) every factor equals one, and the data are projected onto all directions $\boldsymbol{u}_j$ alike. Ridge multiplies each direction by the shrinkage factor $s_j^2/(s_j^2 + n\lambda)$, which is close to one for $s_j^2 \gg n\lambda$ and close to zero for $s_j^2 \ll n\lambda$ [1, 3]. Ridge therefore leaves the well-determined directions nearly untouched and suppresses those with small singular values, which are the directions in which the OLS coefficients are most sensitive to noise. The price is a bias, since part of the signal in those directions is removed as well.

Two quantities summarise this effect. The *effective number of degrees of freedom*,

$$\mathrm{df}(\lambda) = \sum_{j=1}^{p}\frac{s_j^2}{s_j^2 + n\lambda}, \tag{10}$$

equals p for OLS and decreases towards zero as λ grows; it measures how many parameters the model effectively uses [3]. The penalty also improves the conditioning of the linear system that has to be solved, since

$$\mathrm{cond}(\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I}) = \frac{s_1^2 + n\lambda}{s_p^2 + n\lambda}, \tag{11}$$

which is bounded by roughly $s_1^2/(n\lambda)$ once $n\lambda \gg s_p^2$.

4. The Lasso

Lasso regression is similar to Ridge in the sense that you add a penalty; however, the 2-norm is replaced with a 1-norm:

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^p}\left\{\frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_1\right\}. \tag{12}$$

A consequence of this is that Lasso sets coefficients exactly to zero, unlike Ridge [1]. Since $|\theta_j|$ is not differentiable at $\theta_j = 0$, the Lasso has no closed-form solution in general; we solve it with gradient methods as described in Section II L.

C. Data generation and preprocessing

We draw $n = 100$ points $x_i \sim \mathcal{U}[-1, 1]$ and generate the targets according to (2), with f the Runge function (1) and noise level $\sigma = 0.1$. All random numbers are generated with the seed 2026. The data are split at random into a training set (70 %) and a test set (30 %) *before* any

preprocessing. This is within the common rule of thumb of using between 2/3 and 4/5 of the data for training.

For a polynomial of degree d (so $p = d + 1$ parameters including the intercept), the feature matrix contains the columns $x, x^2, \ldots, x^d$, without a column of ones. Each column is standardised by subtracting its mean and dividing by its standard deviation, and the targets are centred by subtracting their mean $\bar{y}_{\text{train}}$. All these statistics are computed from the training set only and then applied unchanged to the test set, so that no information from the test data leaks into the model. Since the centred training data have zero mean, the intercept no longer has to be estimated as a parameter; predictions are made as

$$\tilde{y}(x) = \bar{y}_{\text{train}} + \sum_{j=1}^{d} \theta_j \, \frac{x^j - \mu_j}{\delta_j}, \tag{13}$$

where μ_j and δ_j are the training mean and standard deviation of the j-th column, respectively.

Scaling deserves a critical comment. On $[-1, 1]$ all monomials are bounded by one, but their spread shrinks rapidly with the power, so the unscaled columns have very different magnitudes. For OLS this does not matter for the predictions: with an intercept in the model, standardisation is an invertible affine reparametrisation of the column space, so the fitted values are identical and only the coefficients change. We verify this numerically in Section III A. Scaling does matter for Ridge and Lasso, where the penalty acts on all coefficients equally and therefore depends on the units of each feature, and for gradient descent, where it affects the conditioning of the Hessian. We centre rather than penalise the intercept, since shrinking θ_0 towards zero would bias the model towards $\tilde{y} = 0$ for no reason. Standardisation does not remove the strong correlation between neighbouring powers such as x^j and x^{j+2}, so the feature matrix remains ill-conditioned at high degree.

D. Evaluation metrics

Besides the MSE (4), we use the coefficient of determination

$$R^2(\boldsymbol{y}, \tilde{\boldsymbol{y}}) = 1 - \frac{\sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2}{\sum_{i=0}^{n-1} (y_i - \bar{y})^2}. \tag{14}$$

$R^2 = 1$ corresponds to a perfect fit, and $R^2 = 0$ to a model that does no better than predicting the mean; on test data R^2 can be negative. Since the noise is irreducible, even the true function f would give an expected test MSE of $\sigma^2 = 10^{-2}$, which serves as a lower reference for all our results.

E. Bootstrap

The fitted model depends on the particular training set we happen to have. To see how much, one would ideally repeat the experiment with many independent training sets drawn from the same distribution, but we only have one. The bootstrap approximates this by resampling [1, 3]. From the n_{train} training points, we draw n_{train} points *with replacement*, which gives a new training set of the same size in which some points appear several times and others not at all. We fit the model to each of B such resamples and evaluate it on the same, fixed test set, which gives B predictions $\tilde{y}_i^{(b)}$, $b = 1, \ldots, B$, at every test point x_i. The scaler is refitted on every resample, so that no information from the rest of the data enters the fit. Expectation values over training sets are then approximated by averages over b. We use $B = 300$. Unlike analytical expressions for the variance of the OLS estimator, this requires no assumption about the distribution of the noise.

Since the inputs are drawn at random, some test points may fall outside the range spanned by the training inputs. A prediction there is an extrapolation of the polynomial, which is known to be unstable near the end points of the interval for the Runge function (Section II A). We therefore compute all bootstrap quantities both over all test points and over the *interior* test points, those that lie inside the range of the training inputs.

F. Cross-validation

A single train/test split uses each data point only once, either for training or for testing, and the resulting test error depends strongly on which points end up in the test set. With few data points this makes the estimate noisy, as we will see in Section III A. In k-fold cross-validation [1, 3], the data are shuffled and divided into k folds of (nearly) equal size. For each fold in turn, the model is trained on the other $k - 1$ folds and evaluated on the held-out fold. The cross-validated estimate of the test error is the average of the k fold errors,

$$\text{MSE}_{\text{CV}} = \frac{1}{k} \sum_{m=1}^{k} \text{MSE}_m. \tag{15}$$

Every point is thus used for testing exactly once and for training $k - 1$ times. The training sets contain a fraction $(k-1)/k$ of the data, 80 % for $k = 5$ and 90 % for $k = 10$. Larger k gives training sets closer in size to the full data set, at the price of more fits and fold errors that are more strongly correlated. The polynomial degree, and for Ridge the penalty λ, are then chosen as the values that minimise MSE_{CV}.

We apply cross-validation to all $n = 100$ points with $k = 5$ and $k = 10$, using the same fold assignment (`KFold`, seed 2026) for all models. Any preprocessing that learns from the data, here the standardisation of $\boldsymbol{X}$ and the centring of $\boldsymbol{y}$, must be fitted on the training folds only. Otherwise the held-out points influence the transformation of the training data, the model is no longer evaluated

on unseen data, and the error estimate is biased. We implement k-fold cross-validation with our own code and with Scikit-Learn's `cross_val_score`, where the scaling is placed in a `Pipeline` so that it is refitted in every fold; for Ridge, `alpha` $= n_{\text{train}}\lambda$.

G. The bias-variance tradeoff

Consider a fixed test point x_i with $y_i = f_i + \varepsilon_i$, where $f_i = f(x_i)$, and let $\tilde{y}_i$ be the prediction at x_i of a model trained on a random training set $\mathcal{L}$. The expectation $\mathbb{E}$ below is taken over both the training set and the noise ε_i at the test point. These are independent, since the test point is not used in the training. With $\bar{y}_i = \mathbb{E}[\tilde{y}_i]$ denoting the average prediction over training sets, we write

$$y_i - \tilde{y}_i = (f_i - \bar{y}_i) + (\bar{y}_i - \tilde{y}_i) + \varepsilon_i. \tag{16}$$

When this is squared, the three cross terms vanish in expectation. The two terms containing ε_i vanish because $\mathbb{E}[\varepsilon_i] = 0$ and ε_i is independent of $\tilde{y}_i$, and the remaining one vanishes because $f_i - \bar{y}_i$ is a constant and $\mathbb{E}[\bar{y}_i - \tilde{y}_i] = 0$. Hence

$$\mathbb{E}\left[(y_i - \tilde{y}_i)^2\right] = (f_i - \bar{y}_i)^2 + \mathbb{E}\left[(\tilde{y}_i - \bar{y}_i)^2\right] + \sigma^2. \tag{17}$$

Averaging over the n test points gives the bias-variance decomposition

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \text{Bias}^2[\tilde{y}] + \text{var}[\tilde{y}] + \sigma^2, \tag{18}$$

with

$$\text{Bias}^2[\tilde{y}] = \frac{1}{n}\sum_i (f_i - \bar{y}_i)^2, \tag{19}$$

$$\text{var}[\tilde{y}] = \frac{1}{n}\sum_i \mathbb{E}\left[(\tilde{y}_i - \bar{y}_i)^2\right], \tag{20}$$

see also [1, 3]. The squared bias measures how far the average prediction is from the true function, and is large for models that are too simple to represent f. The variance measures how much the prediction changes from one training set to another, and is large for flexible models that adapt to the noise in the particular training set. The noise variance σ^2 is irreducible: no model can predict the noise at a new point. Increasing the model complexity typically lowers the bias and raises the variance, so the test error has a minimum at an intermediate complexity.

Since f is unknown in practice, the bias is usually computed with the data y_i in place of f_i:

$$\begin{aligned}\text{Bias}_y^2 &= \frac{1}{n}\sum_i (y_i - \bar{y}_i)^2 \\ &= \text{Bias}^2[\tilde{y}] + \frac{1}{n}\sum_i \varepsilon_i^2 + \frac{2}{n}\sum_i \varepsilon_i (f_i - \bar{y}_i).\end{aligned} \tag{21}$$

In expectation over the noise, the last term vanishes and the middle term equals σ^2, so $\mathbb{E}[\text{Bias}_y^2] = \text{Bias}^2 + \sigma^2$: the measured bias absorbs the noise variance. The decomposition then reads error $= \text{Bias}_y^2 + \text{var}$, without a separate noise term, and the measured bias can never fall much below σ^2, even for a perfect model. For a finite test set, the cross term in (21) does not vanish exactly. Since our data are synthetic, we know f and can compute the bias both ways.

In practice we estimate $\bar{y}_i$ by the average over the B bootstrap predictions, and the error and the variance by the corresponding averages over test points and resamples. With these sample averages, error $= \text{Bias}_y^2 + \text{var}$ holds exactly, which we use as a check of the code.

H. Gradient descent

OLS and Ridge have closed-form solutions, but most machine-learning models do not, and their parameters must be found iteratively. We study iterative methods on OLS and Ridge, where the closed form provides an exact reference. Newton's method, $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{H}^{-1}\nabla C(\boldsymbol{\theta}_k)$ with the Hessian $\boldsymbol{H}$, costs $\mathcal{O}(p^3)$ operations per step. *Gradient descent* replaces $\boldsymbol{H}^{-1}$ by a scalar *learning rate* $\eta > 0$ [1],

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta\,\nabla C(\boldsymbol{\theta}_k). \tag{22}$$

For the Ridge cost (7), with OLS as the special case $\lambda = 0$, the gradient and the Hessian are

$$\nabla C(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta}, \tag{23}$$

$$\boldsymbol{H} = \frac{2}{n}\boldsymbol{X}^T\boldsymbol{X} + 2\lambda\boldsymbol{I}. \tag{24}$$

The Hessian is constant and symmetric positive definite, with eigenvalues $0 < h_{\min} \le h_i \le h_{\max}$. (We write h_i rather than λ_i to avoid confusion with the Ridge penalty.) Since the cost is quadratic, the error $\boldsymbol{e}_k = \boldsymbol{\theta}_k - \boldsymbol{\theta}^*$ relative to the minimum $\boldsymbol{\theta}^*$ obeys $\boldsymbol{e}_{k+1} = (\boldsymbol{I} - \eta\boldsymbol{H})\boldsymbol{e}_k$ exactly. Expanding $\boldsymbol{e}_k$ in the eigenvectors of $\boldsymbol{H}$, each component is multiplied by $1 - \eta h_i$ in every step. Gradient descent therefore converges from every starting point if and only if $|1 - \eta h_i| < 1$ for all i, that is

$$0 < \eta < \frac{2}{h_{\max}}, \tag{25}$$

and the error decreases at least as fast as $\|\boldsymbol{e}_k\| \le \rho^k \|\boldsymbol{e}_0\|$ with $\rho(\eta) = \max_i |1 - \eta h_i|$ [1]. The rate is limited by both ends of the spectrum: the slowest component, with $h_{\min}$, decays with the factor $1 - \eta h_{\min}$, while η cannot exceed $2/h_{\max}$. With $\eta = 1/h_{\max}$ we get $\rho = 1 - 1/\kappa$, where $\kappa = h_{\max}/h_{\min}$ is the condition number of the Hessian, and the best fixed learning rate, $\eta_{\text{opt}} = 2/(h_{\max} + h_{\min})$, gives $\rho = (\kappa - 1)/(\kappa + 1)$. In both cases, reducing the error by a factor ϵ takes of the order of $\kappa \ln(1/\epsilon)$ iterations (or half of that for η_{opt}). For the ill-conditioned polynomial

feature matrices of this project, with condition numbers up to 10^{10} (Section III A), this is the central limitation of gradient descent. Note that κ for OLS equals the condition number of $\boldsymbol{X}^T\boldsymbol{X}$, and for Ridge that of $\boldsymbol{X}^T\boldsymbol{X}+n\lambda\boldsymbol{I}$, which we computed in (11).

I. Automatic differentiation

We compute the gradient (23) both analytically and by *automatic differentiation* (AD) with JAX [4] in double precision: the cost is written as an ordinary Python function, and the library returns a function that evaluates its gradient. AD is neither symbolic nor numerical differentiation [1]. Symbolic differentiation manipulates expressions to produce a formula for the derivative, which can grow rapidly in size. Numerical differentiation approximates the derivative by finite differences, which needs $2p$ evaluations of C for a central-difference gradient and suffers from truncation and cancellation errors. AD instead applies the chain rule to the numerical values of the elementary operations that the program performs when it evaluates C, which gives the exact derivative of the computed function up to round-off. In *reverse mode*, used by `jax.grad`, the cost is evaluated once while the intermediate values are stored, and the derivatives are then propagated backwards once. The full gradient therefore costs a small constant multiple of one evaluation of C, independent of p [5]. This is why reverse-mode AD, known as back-propagation for neural networks, is the standard tool in machine learning.

J. Momentum and adaptive learning rates

Plain gradient descent uses one learning rate for all directions, and as shown in Section II H this rate is limited by the stiffest direction while the convergence is limited by the flattest. The methods below modify the update (22) to reduce this problem [1]. All of them use the gradient $\boldsymbol{g}_k = \nabla C(\boldsymbol{\theta}_k)$ only, so they work equally well with the analytical gradient and with automatic differentiation. Squares, square roots and divisions of vectors below are taken element by element.

a. Momentum. Gradient descent with momentum (the *heavy-ball* method) accumulates a velocity $\boldsymbol{v}$,

$$\boldsymbol{v}_{k+1} = \gamma \boldsymbol{v}_k + \eta\, \boldsymbol{g}_k, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \boldsymbol{v}_{k+1}, \tag{26}$$

with $0 \le \gamma < 1$ [1]. In directions where the gradient keeps its sign, the steps add up and the effective learning rate approaches $\eta/(1-\gamma)$; in directions where it alternates, the contributions partly cancel, which damps oscillations. For a quadratic cost the iteration can be analysed eigendirection by eigendirection, as for plain gradient descent. It is stable for $0 < \eta < 2(1+\gamma)/h_{\max}$, so with $\gamma = 0.9$ the admissible learning rates are almost twice as large as for plain gradient descent. With the optimal parameters

$$\eta^* = \frac{4}{\left(\sqrt{h_{\max}} + \sqrt{h_{\min}}\right)^2}, \qquad \gamma^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2, \tag{27}$$

the error decreases asymptotically by the factor $(\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ per step [6]. The number of iterations then scales as $\sqrt{\kappa}\ln(1/\epsilon)$ instead of $\kappa\ln(1/\epsilon)$, a large gain for ill-conditioned problems. In practice $h_{\min}$ and $h_{\max}$ are rarely known, and $\gamma = 0.9$ is a common default.

b. AdaGrad. The adaptive methods give each parameter its own learning rate, based on the history of its gradient component. AdaGrad accumulates the squared gradients,

$$\boldsymbol{G}_{k+1} = \boldsymbol{G}_k + \boldsymbol{g}_k^2, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta\, \boldsymbol{g}_k}{\sqrt{\boldsymbol{G}_{k+1}} + \epsilon}, \tag{28}$$

so that parameters with large past gradients take smaller steps [1]. Since $\boldsymbol{G}$ only grows, the effective learning rate decreases monotonically, which guarantees that the steps eventually become small but can also stop the progress long before the minimum is reached.

c. RMSprop. RMSprop replaces the sum by an exponentially decaying average,

$$\boldsymbol{s}_{k+1} = \rho\, \boldsymbol{s}_k + (1-\rho)\, \boldsymbol{g}_k^2, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \frac{\eta\, \boldsymbol{g}_k}{\sqrt{\boldsymbol{s}_{k+1}} + \epsilon}, \tag{29}$$

with $\rho = 0.99$, so that the effective learning rate follows the recent size of the gradient instead of decaying for good [1].

d. Adam. Adam combines a momentum-like average of the gradient with the RMSprop scaling,

$$\begin{aligned} \boldsymbol{m}_{k+1} &= \beta_1 \boldsymbol{m}_k + (1-\beta_1)\, \boldsymbol{g}_k, \\ \boldsymbol{s}_{k+1} &= \beta_2 \boldsymbol{s}_k + (1-\beta_2)\, \boldsymbol{g}_k^2, \\ \boldsymbol{\theta}_{k+1} &= \boldsymbol{\theta}_k - \eta\, \frac{\hat{\boldsymbol{m}}_{k+1}}{\sqrt{\hat{\boldsymbol{s}}_{k+1}} + \epsilon}, \end{aligned} \tag{30}$$

where $\hat{\boldsymbol{m}}_k = \boldsymbol{m}_k/(1-\beta_1^k)$ and $\hat{\boldsymbol{s}}_k = \boldsymbol{s}_k/(1-\beta_2^k)$ correct for the initialisation at zero [1, 7]. We use the standard values $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$.

e. Limitations. Two properties of RMSprop and Adam matter for our results. Since their steps are normalised by the root-mean-square gradient, they remain of order η near the minimum, so with a constant η the iterate tends to hover around the minimum rather than converge to high precision. And since the rescaling acts on each parameter separately, it is a diagonal preconditioner, which cannot remove ill-conditioning caused by correlated features, whose stiff and flat directions mix many parameters.

We compare plain gradient descent, momentum ($\gamma = 0.9$), AdaGrad, RMSprop and Adam for OLS and Ridge ($\lambda = 10^{-3}$) at degree 5, starting from $\boldsymbol{\theta}_0 = \boldsymbol{0}$ with the analytical gradient and a budget of 2×10^4 iterations. For 21 learning rates from 10^{-4} to 10, we record the

first iteration at which the relative parameter error $\|\boldsymbol{\theta}_k - \boldsymbol{\theta}^*\|/\|\boldsymbol{\theta}^*\|$ falls below 10^{-6}, and whether it stays there. The range of learning rates for which a method reaches and stays below this tolerance measures its sensitivity to η.

K. Stochastic gradient descent

In stochastic (mini-batch) gradient descent, the gradient is computed on a random subset $\mathcal{B}$ of B training points [1],

$$\boldsymbol{g}_{\mathcal{B}}(\boldsymbol{\theta}) = \frac{2}{B}\boldsymbol{X}_{\mathcal{B}}^T(\boldsymbol{X}_{\mathcal{B}}\boldsymbol{\theta} - \boldsymbol{y}_{\mathcal{B}}) + 2\lambda\boldsymbol{\theta}. \tag{31}$$

In each epoch, the training set is shuffled and divided into $\lceil n/B \rceil$ mini-batches, with one update per mini-batch; $B = n$ is full-batch gradient descent. All update rules above are used unchanged. The mini-batch gradient is an unbiased but noisy estimate of the full gradient. Since the noise does not vanish at the minimum, a constant learning rate leaves the excess cost at a *noise floor* that grows with η, and convergence requires a decreasing learning rate with $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$ [1]. We use

$$\eta_t = \frac{\eta_0}{1 + t/T}, \tag{32}$$

where t counts the updates and T is the number of updates in 20 epochs. In addition, the largest stable learning rate is set by the curvature of a single mini-batch, which for $B = 1$ is $2\|\boldsymbol{x}_i\|^2$ and can be much larger than $h_{\max}$.

We study $B = 1, 5, 14, 35$ and 70 (full batch) at degree 5 for OLS and Ridge ($\lambda = 10^{-3}$) for 500 epochs, each with the best constant learning rate among 0.01 to 1 times $1/h_{\max}$, and then constant versus decaying learning rates and all update rules for $B = 5$. Every curve is the median over five shuffling seeds, and accuracy is measured by the relative excess training cost $(C(\boldsymbol{\theta}) - C^*)/C^*$.

L. Lasso with gradient methods

The penalty $\lambda\|\boldsymbol{\theta}\|_1$ is not differentiable where $\theta_j = 0$. There the derivative is replaced by the *subdifferential*, the set of slopes of all lines that touch the function from below, which for $|\theta|$ at zero is $[-1, 1]$; any element of it is a *subgradient* [1, 3]. At the minimum, zero lies in the subdifferential, which for the Lasso means $|\frac{2}{n}\boldsymbol{x}_j^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}^*)| \le \lambda$ for every j with $\theta_j^* = 0$: sufficiently weak features get exactly zero coefficients. Our analytical subgradient is

$$\boldsymbol{g}(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + \lambda\,\mathrm{sign}(\boldsymbol{\theta}), \tag{33}$$

with $\mathrm{sign}(0) = 0$, and all methods above can then be used unchanged. Such *subgradient methods* have a known weakness: a coefficient whose optimal value is zero is pushed by $\eta\lambda$ towards zero in each step, overshoots and is pushed back, so it oscillates with an amplitude of order $\eta\lambda$ and never becomes exactly zero. Which value automatic differentiation returns at the kink depends on the library and must be checked.

As a reference we use the proximal gradient method ISTA [8], which takes a gradient step on the smooth part and then applies the soft-thresholding operator S_t to each component,

$$\boldsymbol{\theta}_{k+1} = S_{\eta\lambda}\Big(\boldsymbol{\theta}_k - \eta\,\tfrac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta}_k - \boldsymbol{y})\Big), \tag{34}$$

$$S_t(z) = \mathrm{sign}(z)\max(|z| - t, 0), \tag{35}$$

Soft thresholding sets all components with $|z| \le t$ exactly to zero, so it handles the kink exactly; with $\eta \le 1/h_{\max}$, where $h_{\max}$ is the largest eigenvalue of the Hessian $\frac{2}{n}\boldsymbol{X}^T\boldsymbol{X}$ of the smooth part, ISTA converges to the Lasso solution. Scikit-Learn's `Lasso` minimises our cost divided by two, so our λ corresponds to $\alpha = \lambda/2$; we use it as a reference with `fit_intercept=False`, a tolerance of 10^{-12} and up to 10^6 iterations.

M. Final model selection

For the final comparison we keep the 70/30 split and use the test set only once, to evaluate the selected models. All choices are made by cross-validation on the 70 training points: OLS as a function of the degree (1 to 15), and Ridge and Lasso as functions of the degree and λ (15 values from 10^{-7} to 1). We use 5-fold cross-validation repeated three times (15 folds), with the scaler fitted on the training folds, and report $\mathrm{MSE}_{\mathrm{CV}}$ with its standard error (SE), the fold standard deviation divided by $\sqrt{15}$; since the folds overlap, this somewhat underestimates the uncertainty. We select the model with the lowest $\mathrm{MSE}_{\mathrm{CV}}$, and also the simplest model within one SE of it (lowest degree, then largest λ), following the *one-standard-error rule* [3]. The Lasso is solved with Scikit-Learn's coordinate descent, which agrees with our ISTA code to 3.4×10^{-12} (Section III G) and is much faster. Combinations of degree and λ for which any fold did not converge within 10^5 iterations are excluded, since an unconverged iterate acts as an uncontrolled regulariser. Finally, we compute the bootstrap decomposition of Section II G with the true $f(x_{\mathrm{test}})$ as reference, so that the error is exactly $\mathrm{Bias}^2 + \mathrm{var}$, using 200 resamples that are the same for every degree, λ and method.

N. Implementation

All regression methods are implemented in our own Python code (the module `utils.py` in our repository). OLS uses the pseudoinverse computed by `numpy.linalg.pinv`, and Ridge solves the linear system $(\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I})\boldsymbol{\theta} = \boldsymbol{X}^T\boldsymbol{y}$ with `numpy.linalg.solve`

rather than forming the inverse explicitly. Scikit-Learn [2] is used for the train/test split (`train_test_split`), to verify our Ridge implementation against `Ridge`, for the fold assignment in k-fold cross-validation (`KFold`), to verify our own cross-validation code, and as a reference solution for the Lasso (`Lasso`). Gradient descent is implemented in our own code, with the gradient computed either from (23) or by JAX [4]; all iterations start at $\boldsymbol{\theta}_0 = \mathbf{0}$.

III. RESULTS AND DISCUSSION

A. Ordinary least squares

Figure 1 shows the OLS results. The training MSE decreases monotonically with the degree, from 1.00×10^{-1} at degree 1 to 1.42×10^{-2} at degree 15. Almost all of the decrease happens at even degrees (from degree 1 to 2 the training MSE nearly halves, from 2 to 3 it changes by less than 1 %), which reflects the symmetry $f(-x) = f(x)$: the odd powers can only fit the noise. Even at degree 15, the training MSE is well above the value $\sigma^2(1-16/70) \approx 0.8\times10^{-2}$ expected for an unbiased model with 16 parameters, so a substantial bias remains. A global polynomial struggles to reproduce the sharp peak and the flat tails at the same time.

The test MSE is far less regular. It reaches 1.87×10^{-2} at degree 6 and then fluctuates strongly: 6.25×10^{-2} at degree 13, the overall minimum 1.45×10^{-2} ($R^2 = 0.86$) at degree 14, and 4.29×10^{-1} ($R^2 = -3.06$) at degree 15, a clear case of overfitting. A minimum lying between two much worse degrees is most likely an accident of this split with only 30 test points; we test this with the bootstrap and cross-validation in Sections III C and III D. The erratic behaviour coincides with the growing ill-conditioning of the problem. The condition number of the standardised feature matrix grows by about a factor 2.5 per degree, to 1.9×10^5 at degree 15, so that of $\boldsymbol{X}^T\boldsymbol{X}$ reaches about 4×10^{10}. Small changes in the data then cause large changes in the coefficients, which is what a high variance means.

The coefficients are shown in Fig. 14 in Appendix B. Up to degree 5 all of them are smaller than one in magnitude, and the even powers dominate, as expected from the symmetry of f. From degree 8 they grow rapidly, to about 30 at degree 10, 90 at degree 12 and almost 700 at degree 15, and neighbouring coefficients take large values of opposite sign that nearly cancel (at degree 15, for example, $\theta_9 = -501$, $\theta_{11} = 679$ and $\theta_{13} = -468$). The odd coefficients, which should be close to zero, become as large as the even ones. The value of a given coefficient also changes completely when the degree is increased; θ_4, for instance, grows from 0.53 at degree 4 to 34 at degree 15. This is the signature of the near-degenerate directions discussed above: the coefficients are poorly determined individually, even though the fit is not. The dependence on the number of data points is analysed with the bootstrap in Section III C, and the dependence on the noise level is shown in Fig. 15 in Appendix B; for low amount of noise, the test MSE decreases in a zigzag pattern – increasing slightly for odd degrees before decreasing further for the next (even) degree – up until degree 8, where the zigzag pattern continuous, but in an increasing manner. For noise level $\sigma = 0.2, 0.5$, the test MSE is mostly stable until around degree 13, where it suddenly increases rapidly. Finally, scaling leaves the OLS predictions unchanged: at degree 10 the predictions from the scaled and the unscaled fit differ by at most 3.25×10^{-14}, which confirms the argument of Section II C.

B. Ridge regression

We repeated the analysis with Ridge for ten values of λ from 10^{-8} to 10. Our implementation agrees with Scikit-Learn's `Ridge` (`alpha`$= n\lambda$, `fit_intercept=False`) to 1.15×10^{-13}. Figure 2 shows that the training MSE increases with λ at every degree, at degree 15 from 1.42×10^{-2} for OLS to 6.56×10^{-2} for $\lambda = 1$; this is the bias introduced by the penalty. Low-degree models are hardly affected. The main effect is at high degree: every λ in the figure removes the jump at degree 15, where the test MSE drops from 0.43 to 1.8×10^{-2} already for $\lambda = 10^{-6}$. A moderate penalty also makes the result insensitive to the degree: for $\lambda = 10^{-3}$ the test MSE lies between 1.99×10^{-2} and 2.25×10^{-2} for all degrees from 6 to 15. At degree 15, the test MSE as a function of λ has the U-shape of the bias-variance tradeoff, with a minimum of 1.51×10^{-2} at $\lambda = 10^{-5}$ and 8.8×10^{-2} at $\lambda = 10$, close to $\mathrm{var}(y) \approx 0.1$.

The lowest test MSE overall is 1.34×10^{-2} (degree 12, $\lambda = 10^{-5}$), against 1.45×10^{-2} for OLS. Both optima are chosen on the test set itself, and at the same λ degree 11 gives almost twice the error, so the difference is not significant; a fair comparison follows in Section III D.

Table I. Effective degrees of freedom (10) and condition number (11) for Ridge at degree 15, computed from the singular values of the standardised training matrix ($n = 70$). $\lambda = 0$ is OLS.

λ	$\mathrm{df}(\lambda)$	$\mathrm{cond}(\boldsymbol{X}^T\boldsymbol{X} + n\lambda\boldsymbol{I})$
0	15.00	3.7×10^{10}
10^{-6}	11.06	9.5×10^{6}
10^{-4}	8.77	9.5×10^{4}
10^{-2}	6.07	9.5×10^{2}
1	2.57	1.0×10^{1}

Table I connects these results to the shrinkage of the singular-value modes (9). At degree 15, the singular values of the standardised training matrix span more than five orders of magnitude, from 25.8 to 1.3×10^{-4}. Already $\lambda = 10^{-6}$ ($n\lambda = 7\times10^{-5}$) damps the four smallest modes, which lowers the effective number of degrees of freedom from 15 to 11 and the condition number from 3.7×10^{10} to 9.5×10^6; from there the condition number

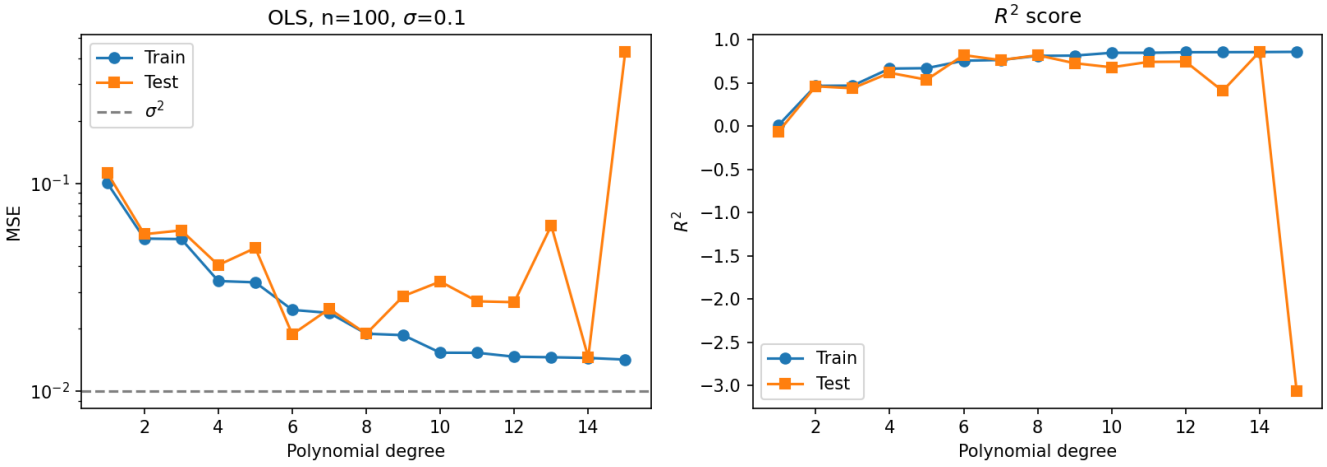


Figure 1. Training and test MSE (left, logarithmic axis) and R^2 (right) for OLS as functions of polynomial degree, for $n = 100$ points of the Runge function with noise $\sigma = 0.1$, a 70/30 train/test split and seed 2026. Features are standardised and targets centred with training statistics. The dashed line marks the noise level $\sigma^2 = 10^{-2}$. The full table of scores and condition numbers is available in `results/output/a/scores_vs_degree.csv` in our repository.

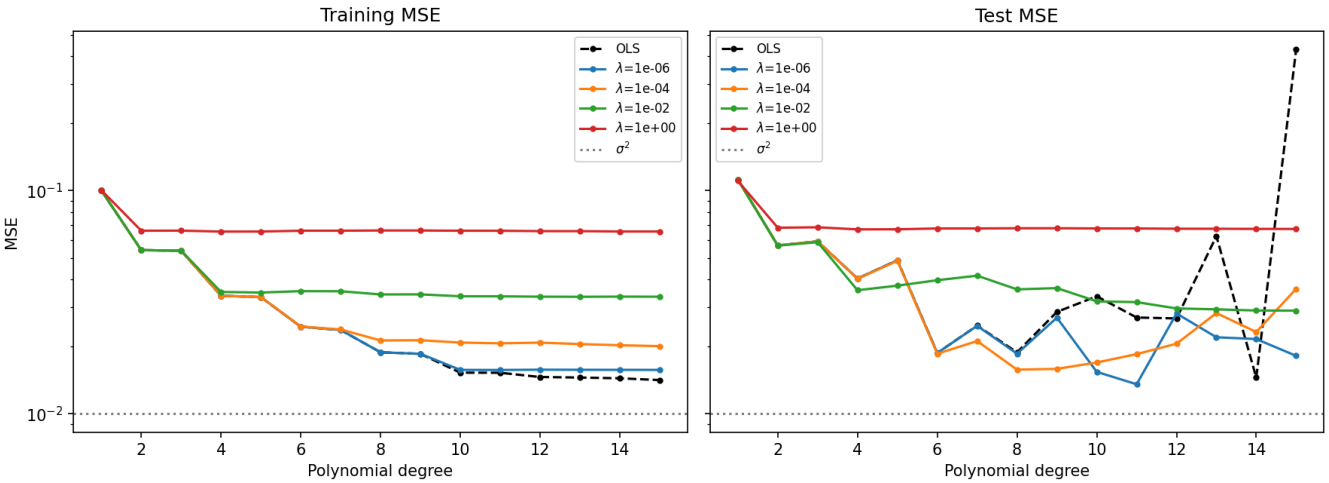


Figure 2. Training (left) and test (right) MSE on a logarithmic axis as functions of polynomial degree for OLS (dashed black) and Ridge regression with $\lambda = 10^{-6}, 10^{-4}, 10^{-2}$ and 1, for $n = 100$ points of the Runge function with noise $\sigma = 0.1$, a 70/30 train/test split and seed 2026. Features are standardised and targets centred with training statistics. The dotted line marks $\sigma^2 = 10^{-2}$.

falls as $s_1^2/(n\lambda)$. At the optimal $\lambda = 10^{-5}$, the five modes with $s_j^2 < n\lambda$ are suppressed. Even $\lambda = 10^{-8}$ damps the two smallest modes, enough to reduce the test MSE at degree 15 from 0.43 to 6.4×10^{-2}. A small penalty thus removes precisely the near-degenerate combinations of monomials behind the ill-conditioning, and leaves the well-determined part of the fit intact.

C. The bias-variance tradeoff

Figure 3 shows the training and test MSE as functions of the model complexity for four sample sizes, as in Fig. 2.11 of Hastie et al. [3]. The training MSE decreases monotonically in every panel, and for small n it falls below the noise level: for $n = 50$, with 35 training points, it is 7.8×10^{-3} at degree 13 and 6.0×10^{-3} at degree 20, a clear sign that the model fits the noise. The test MSE is far from the smooth U-shape of Hastie et al. For $n = 50$ it falls to 9.7×10^{-3} at degree 10, below σ^2, which is possible only by chance with 15 test points, and then explodes: 7.6×10^{-2} at degree 11, 0.47 at degree 12, 14 at degree 13 and more than 10^5 at degrees 19 and 20. For $n = 100$ the test MSE stays erratic but moderate up to degree 14 (Section III A) and rises from degree 15. Instead of a gradual increase, the test error thus grows by orders of

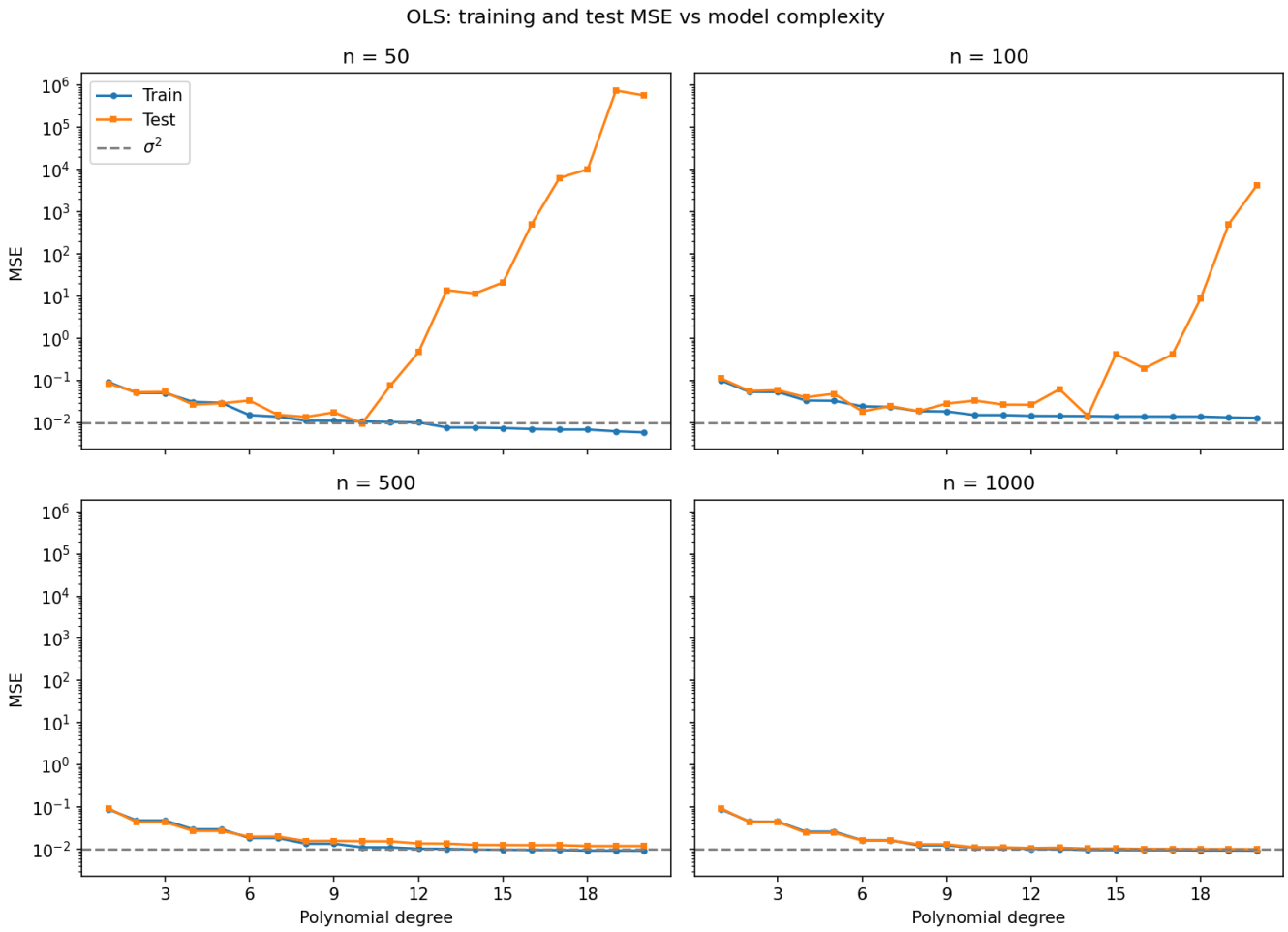


Figure 3. Training and test MSE for OLS as functions of polynomial degree (up to 20) for $n = 50, 100, 500$ and 1000 data points, with $\sigma = 0.1$, a 70/30 train/test split and seed 2026, in the style of Fig. 2.11 of Hastie et al. [3]. A single train/test split is used for each n. The dashed line marks $\sigma^2 = 10^{-2}$.

magnitude within a few degrees once the model starts to extrapolate wildly, and the onset of this growth moves to higher degree as n increases. For $n = 500$ and $n = 1000$ it is not reached within degree 20: both errors decrease almost monotonically, in steps at the even degrees, and the gap between them stays small. For $n = 1000$ the test MSE at degree 20 is 1.005×10^{-2}, essentially the noise level, and the training MSE is 0.94×10^{-2}, slightly below it, as expected for 21 parameters fitted to 700 points. With enough data, the high-variance region of Hastie's figure lies beyond the degrees we have studied.

Figure 4 shows the bootstrap decomposition for $n = 100$. The identity error $= \mathrm{Bias}_y^2 + \mathrm{var}$ holds to a relative accuracy of 4.8×10^{-16}. Over all 30 test points the error has its minimum, 5.29×10^{-2}, already at degree 4. The reason is three test points ($x = -0.974, -0.996, -0.974$) that lie outside the range of the training inputs. There the polynomial must extrapolate near the end of the interval, where the Runge function is hardest to approximate, and high-degree fits vary enormously from one resample to the next. This confirms that the minimum at degree 14 in Section III A was a lucky outcome for one training set. Over the 27 interior test points, the classic tradeoff appears: the error is lowest, 1.88×10^{-2}, at degree 8, where $\mathrm{Bias}_y^2 = 1.39 \times 10^{-2}$ and $\mathrm{var} = 4.97 \times 10^{-3}$. The squared bias falls from 0.100 at degree 1 to about 0.014 at degree 8 and then stays at 0.014–0.016 up to degree 14, while the variance grows steadily, from 1.8×10^{-3} at degree 2 to 5.0×10^{-3} at degree 8, 1.0×10^{-2} at degree 10 and 5.1×10^{-2} at degree 12. The bias thus dominates up to degree 11, and the variance from degree 12, where it jumps to more than three times the bias; at degree 15 it is 2.2. With the true function as reference, the squared bias at degree 8 is only 2.9×10^{-3}, so most of the measured bias at the optimum is noise. Separating the two sets of points is not a way of improving the numbers, but of separating interpolation from extrapolation, where polynomials are notoriously unreliable. For the interior points, $\mathrm{Bias}_y^2 - \mathrm{Bias}_f^2$ lies between 1.09×10^{-2} and 2.59×10^{-2} (degrees 1–8), against a realised noise

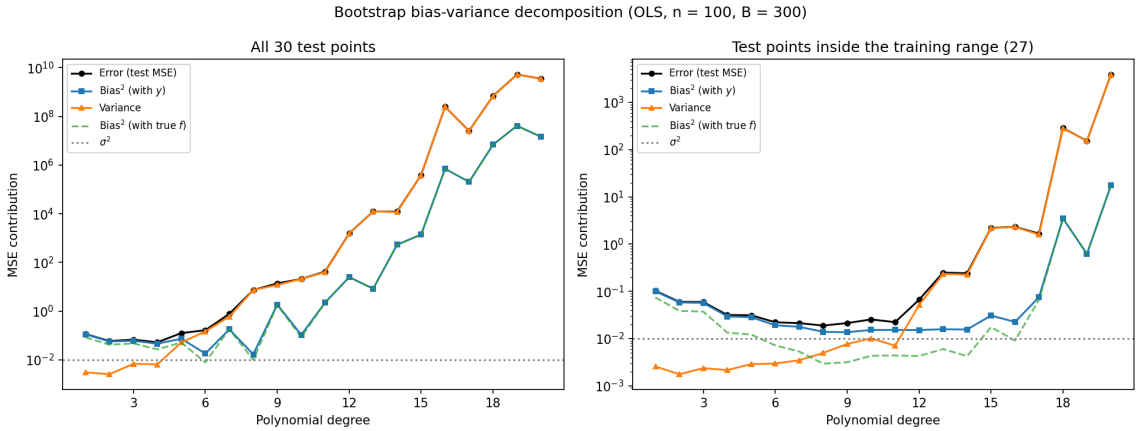


Figure 4. Bootstrap bias-variance decomposition of the test error for OLS as a function of polynomial degree, for $n = 100$, $\sigma = 0.1$ and $B = 300$ bootstrap resamples of the 70 training points. Left: all 30 test points. Right: the 27 interior test points, which lie inside the range of the training inputs. Shown are the error (test MSE), the squared bias computed with the data y and with the true function f, and the variance. The dotted line marks $\sigma^2 = 10^{-2}$.

variance of 9.99×10^{-3}. The measured bias thus absorbs the noise, as derived in (21); the excess comes from the cross term, which does not cancel for 27 points.

Repeating the analysis for $n = 50$, 100, 500 and 1000, the optimal degree (minimum interior error) grows from 2 to 8, 19 and 20. With 35 training points ($n = 50$), any flexible model has so much variance that the best compromise is a simple and strongly biased one ($\mathrm{Bias}_y^2 = 4.4 \times 10^{-2}$). As n grows, the variance at the optimum falls, to 3.4×10^{-4} for $n = 1000$, which allows more complex models with lower bias. For $n = 1000$ the error is within 5 % of σ^2 (at degree 20, the edge of our range), and no test points lie outside the training range. More data thus shift the optimal complexity upwards, as in Hastie et al. [3]. The full decomposition for each n is available in our repository (`bias_variance_vs_n.png`).

D. Cross-validation

Our own k-fold code agrees with Scikit-Learn to 10^{-13} in relative terms for degrees up to 12 (up to 3×10^{-9} at higher degrees, where the ill-conditioning amplifies round-off), and to 3.6×10^{-12} for Ridge. Figure 5 shows the cross-validated MSE for OLS. The minimum lies at degree 12 for both $k = 5$ (2.02×10^{-2}) and $k = 10$ (1.73×10^{-2}). Above degree 12 the curves become erratic, and the spread over the folds shows that a single fold is responsible: for $k = 5$, the fold standard deviation is 1.8×10^{-3} at degree 12 and 0.41 at degree 17. That fold contains one of the two most extreme inputs ($x = -0.9964$ and 0.9795), which then requires extrapolation. If we keep these two points for training but leave them out of the error average, the $k = 10$ curve has a broad minimum between 1.73×10^{-2} and 1.91×10^{-2} for all degrees from 10 to 18. Estimating the standard error as the fold standard deviation divided by $\sqrt{k}$, degrees 8 to 14 cannot be distinguished; for $k = 10$ we get, for example, $(1.73 \pm 0.13) \times 10^{-2}$ at degree 12 and $(1.93 \pm 0.21) \times 10^{-2}$ at degree 13.

The bootstrap pointed to degree 4 (all points) and 8 (interior points), and cross-validation to degree 12 (10 without the extreme inputs), with similar errors at the optima. Cross-validation thus favours more complex models and is much more tolerant of high degrees: at degree 15, the interior bootstrap error is 2.2, while the cross-validation error without the extreme inputs is 1.9×10^{-2}. Three differences explain this. First, in the bootstrap every model has to extrapolate to the three exterior test points, whereas in cross-validation only two points ever require extrapolation, each in one fold. Second, the training sets in cross-validation are larger (80 or 90 points, against 70), and the optimal degree grows with n. Third, a bootstrap resample contains only about $1 - e^{-1} \approx 63\,\%$ distinct points and often lacks the points nearest to the edges, which raises the variance of flexible models; the simple bootstrap is therefore pessimistic for them [3, Sec. 7.11]. Both methods agree that the best OLS models have degrees of roughly 8 to 12 and an error about twice the noise level.

For Ridge, cross-validation selects degree 11 with $\lambda = 10^{-6}$ (1.79×10^{-2}, $k = 5$) and degree 12 with $\lambda = 10^{-8}$ (1.76×10^{-2}, $k = 10$), within the uncertainty of OLS: the advantage found on the single split in Section III B is not robust. Ridge does, however, remove the explosion at high degree. With $\lambda = 10^{-4}$ and $k = 5$, the error stays between 2.2×10^{-2} and 3.1×10^{-2} for all degrees from 6 to 20, against 1.8×10^{3} for OLS at degree 20.

Finally, fitting the scaler on all data instead of inside the loop changes the cross-validated Ridge error by between 3×10^{-4} and 6×10^{-2} in relative terms, without a consistent sign. With these few, simple transformations the leakage is smaller than the noise in the estimate, but only scaling inside the loop gives an estimate on truly

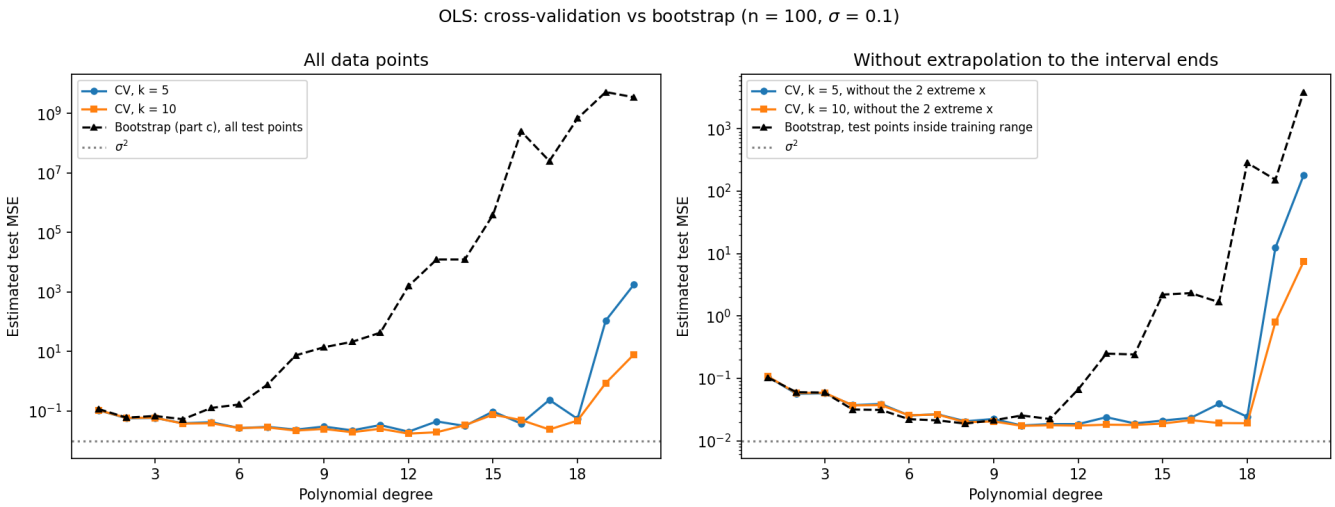


Figure 5. Estimated test MSE for OLS as a function of polynomial degree, from k-fold cross-validation ($k = 5$ and 10) on all $n = 100$ points and from the bootstrap of Section III C ($B = 300$, 70/30 split), on a logarithmic axis. Left: all data points. Right: without extrapolation, i.e. cross-validation evaluated on all points except the two most extreme inputs ($x = -0.9964$ and 0.9795), and the bootstrap over the 27 interior test points. $\sigma = 0.1$, seed 2026; scaling is refitted on the training data in every fold and resample. The dotted line marks $\sigma^2 = 10^{-2}$.

unseen data.

E. Gradient descent

We use degree 5 and, for Ridge, $\lambda = 10^{-3}$, and regard the iteration as converged when the relative parameter error falls below 10^{-6}. At random points, the JAX gradient agrees with the analytical gradient to 2–5 $\times\, 10^{-16}$ for degrees 5, 10 and 15, while central finite differences with $h = 10^{-6}$ deviate by about 10^{-10}. Since the cost is quadratic, this deviation is entirely round-off. The two gradients give identical runs, so we show results for the analytical one.

Figure 6 shows that gradient descent converges to the closed-form solutions, with test MSEs that agree to six digits (4.8778×10^{-2} for OLS). With $\kappa = 518$, OLS needs 6602 iterations with $\eta = 1/h_{\max}$ and 3309 with η_{opt}, 7–8 % below the bounds $\ln(10^{-6})/\ln\rho$ of 7143 and 3575 iterations, which assume that the whole initial error lies in the slowest direction. Ridge raises $h_{\min}$, which lowers κ to 447 and the iteration counts by about 15 %. Since η_{opt} lies just below $2/h_{\max}$, the stiffest error component changes sign in every step, so the test MSE oscillates for many iterations. A scan of 17 learning rates confirmed the stability limit (25): gradient descent converged for $\eta \le 0.999 \times 2/h_{\max}$ and diverged for $\eta \ge 1.001 \times 2/h_{\max}$, and the measured iteration counts were 80–92 % of the prediction from $\rho(\eta)$.

Figure 7 shows the consequence of the ill-conditioning. For OLS, κ grows from 5.2×10^2 at degree 5 to 3.7×10^{10} at degree 15. Gradient descent needs 30 576 iterations at degree 6 and does not converge within 10^5 iterations from degree 7. Standardisation lowers κ by less than a factor three, since it cannot remove the correlation between the monomials. A Ridge penalty bounds κ by roughly $h_{\max}/(2\lambda)$: with $\lambda = 10^{-2}$, gradient descent converges at every degree in at most 11 228 iterations, consistent with $\kappa = 9.5 \times 10^2$ at degree 15 (Table I).

Not converging does not mean poor predictions. For OLS at degree 12 ($\kappa = 1.1 \times 10^8$), the relative parameter error is still 0.99 after 10^5 iterations, yet the test MSE is 2.53×10^{-2}, lower than the closed-form 2.68×10^{-2}, and its lowest value along the path is 1.76×10^{-2}, after 12 087 iterations. Starting from zero, the component along an eigendirection with eigenvalue h_i has after k steps reached the fraction $1 - (1 - \eta h_i)^k$ of its final value. The poorly determined directions that carry the variance are therefore barely fitted, which is the same filtering as the shrinkage factors of Ridge: stopping early acts as an implicit regularisation [9, Sec. 7.8]. The minimum is found on the test set and is uncertain, but the result shows that the predictions depend on the well-determined directions, which gradient descent finds quickly.

F. Momentum and adaptive learning rates

Figure 8 shows that the methods fall into three groups. OLS and Ridge behave alike, so we quote OLS. Plain gradient descent converges for only three of the 21 learning rates (0.056 to 0.18): larger values exceed $2/h_{\max} = 0.31$, and smaller ones need more than the 2×10^4 iterations available. Momentum with $\gamma = 0.9$ works from 5.6×10^{-3} to 0.56, where the upper end matches the predicted limit $2(1 + \gamma)/h_{\max} = 0.58$. AdaGrad is the most robust

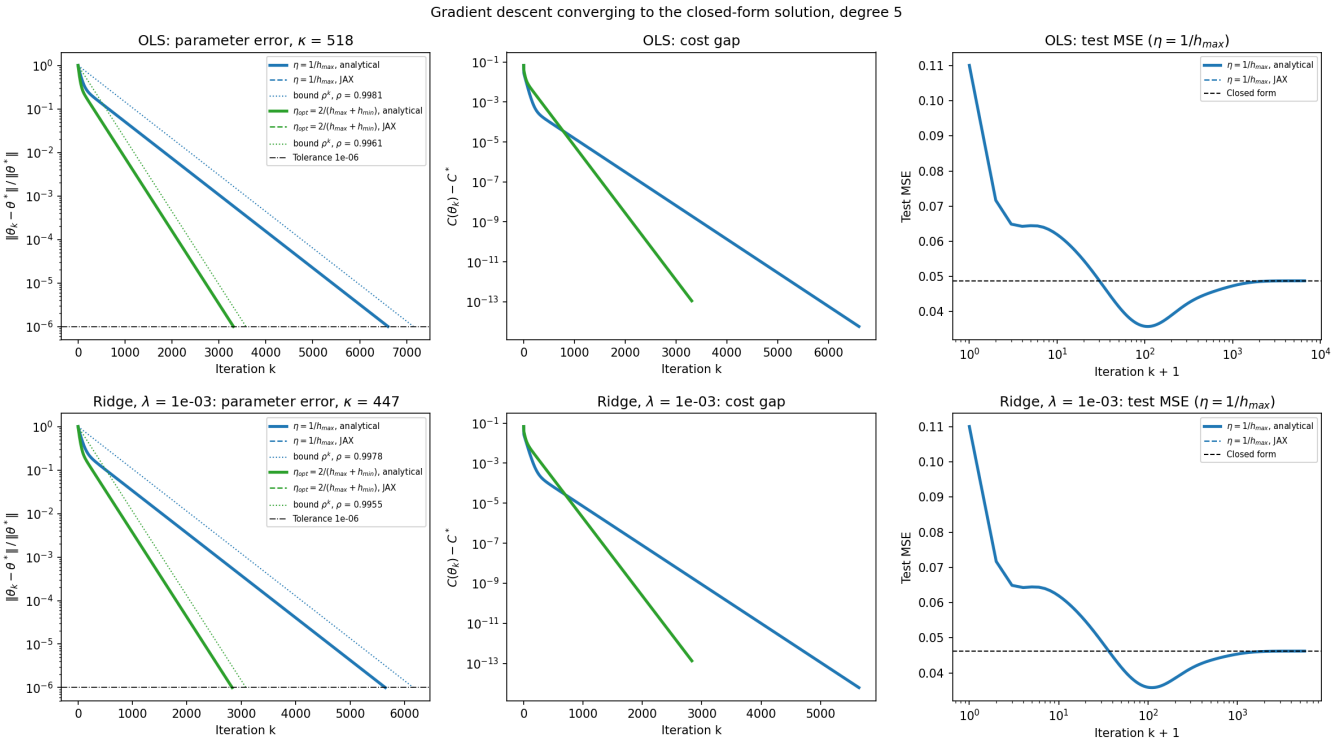


Figure 6. Gradient descent for OLS (top) and Ridge with $\lambda = 10^{-3}$ (bottom) at polynomial degree 5, starting from $\boldsymbol{\theta}_0 = \mathbf{0}$, with $\eta = 1/h_{\max}$ (blue) and $\eta_{\text{opt}} = 2/(h_{\max} + h_{\min})$ (green). Solid lines use the analytical gradient and dashed lines JAX; the two coincide. Left: relative parameter error, with the theoretical bound ρ^k (dotted) and the tolerance 10^{-6}. Middle: cost relative to its minimum. Right: test MSE for $\eta = 1/h_{\max}$, compared with the closed-form solution (dashed). $n = 100$, $\sigma = 0.1$, 70 training points, seed 2026.

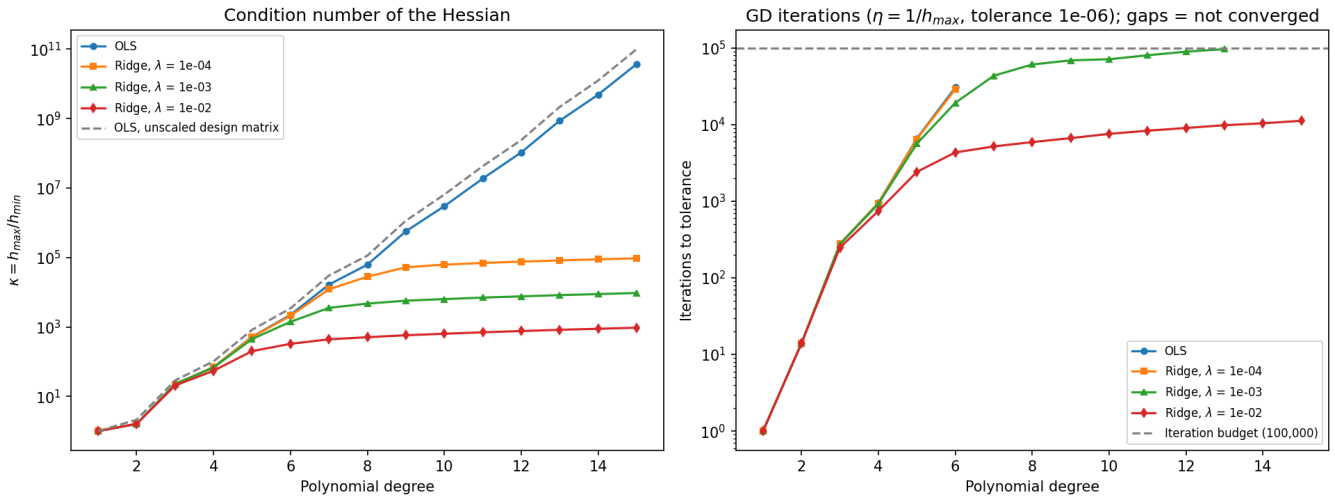


Figure 7. Left: condition number $\kappa = h_{\max}/h_{\min}$ of the Hessian as a function of polynomial degree, for OLS and Ridge with $\lambda = 10^{-4}, 10^{-3}$ and 10^{-2} using standardised features, and for OLS with the unscaled feature matrix (grey). Right: number of gradient descent iterations with $\eta = 1/h_{\max}$ needed to reach a relative parameter error of 10^{-6}, within a budget of 10^5 iterations (dashed line); missing points did not converge. Same data as Fig. 6.

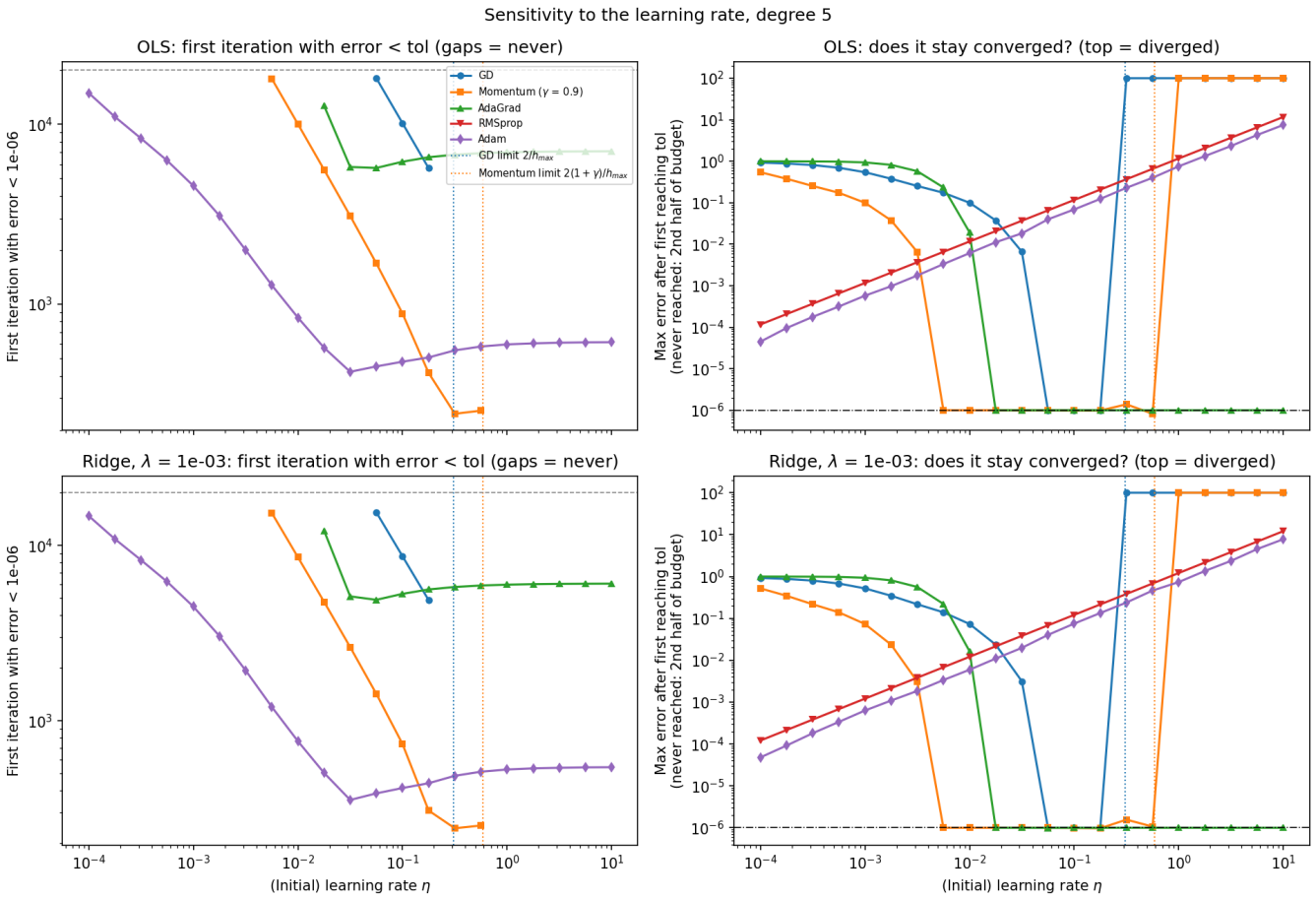


Figure 8. Sensitivity to the (initial) learning rate η for plain gradient descent (GD), momentum ($\gamma = 0.9$), AdaGrad, RMSprop and Adam, for OLS (top) and Ridge with $\lambda = 10^{-3}$ (bottom) at polynomial degree 5, with 21 values of η from 10^{-4} to 10 and a budget of 2×10^4 iterations. Left: first iteration at which the relative parameter error falls below 10^{-6} (missing points: never). Right: largest error after that iteration (or over the second half of the budget if the tolerance is never reached); points below the dash-dotted line stay converged. Dotted vertical lines: stability limits $2/h_{\max}$ for GD and $2(1+\gamma)/h_{\max}$ for momentum. Same data as Fig. 6.

method and converges for every learning rate from 0.018 up to 10, since its growing denominator compensates an overly large learning rate. RMSprop never reaches 10^{-6} (its best error is 1.2×10^{-4}, at the smallest η), and Adam reaches it for all 21 learning rates but never stays there (later errors up to 1.8×10^{-2}). As expected from Section II J, their normalised steps remain of order η near the minimum, so with a constant learning rate they hover around it. The convergence curves at the best learning rates (`convergence.png` for part f in our repository) show how this happens for Adam: its error falls to round-off level after about 10^3 iterations, but after about 1600 iterations it jumps back to about 2×10^{-2} and stays at that level. Near the minimum the gradient, and with it the running average $\boldsymbol{s}$ of its square, decays towards zero with a memory of about $1/(1-\beta_2) = 1000$ iterations; once $\boldsymbol{s}$ has become small, a tiny gradient again gives a step of order η, which throws the iterate away from the minimum. RMSprop with $\eta = 10^{-4}$, in contrast, converges slowly and levels off at 1.2×10^{-4}.

At the best learning rates, plain gradient descent needs 5710 iterations and AdaGrad 5725, while momentum needs 256 and momentum with the optimal parameters (27) 194. The predicted rates give 7143 and 157 iterations, a ratio of 45, close to the measured 34: momentum changes the scaling from κ to $\sqrt{\kappa}$. The ranking reflects the structure of the problem. The ill-conditioning comes from correlated features, so the slow directions mix many parameters. Momentum accelerates every eigendirection, whereas the adaptive methods rescale each parameter separately, which gains little when the features are already standardised. For all methods the test MSE agrees with the closed form to four digits, since the remaining error lies in flat directions. For computing a precise minimiser, momentum is clearly the best method here, while RMSprop and Adam would need a decaying learning rate.

G. Lasso regression

JAX returns 1 for the derivative of $|\theta|$ at $\theta = 0$. This lies in the subdifferential $[-1, 1]$ and is therefore a valid subgradient, but differs from our choice $\operatorname{sign}(0) = 0$: with two components set to zero, the two subgradients differ by exactly λ in those components and agree to machine precision elsewhere. The value at the kink is an implementation convention rather than a property of the cost. In practice it does not matter, because the iterates of a subgradient method practically never land exactly on zero.

Figure 9 shows the central difficulty, at degree 5 with $\lambda = 10^{-2}$, where two of the five coefficients of the solution are zero. None of the subgradient methods reaches a cost gap below 10^{-8} in 10^4 iterations (final gaps between 1.2×10^{-6} and 1.9×10^{-5}), and none sets the two coefficients to zero. For plain gradient descent they oscillate with magnitudes up to 8.5×10^{-4}, of the order of $\eta\lambda = 1.5 \times 10^{-3}$, as predicted in Section II L. ISTA, with the same learning rate, reaches a gap below 10^{-8} after 633 iterations and 1.4×10^{-17} at the end, sets both coefficients exactly to zero, and agrees with Scikit-Learn to 3.4×10^{-12}, which also verifies the conversion $\alpha = \lambda/2$.

Figure 10 contrasts the coefficient paths at degree 8. Scikit-Learn's Lasso has no zero coefficients for $\lambda \leq 3 \times 10^{-4}$, then progressively more, until all eight vanish at $\lambda = 0.6$. Ridge shrinks all coefficients smoothly, and none becomes zero: the ℓ_1 penalty selects features, the ℓ_2 penalty only shrinks them. Our subgradient method produces no exact zeros along the path; counting $|\theta_j| < 2\eta\lambda$ as zero, the counts agree with Scikit-Learn for 17 of the 21 values of λ and differ by one otherwise. The coefficients themselves differ by up to 2.5 where the costs differ by only 9×10^{-4}: with $\kappa \approx 6 \times 10^4$ at degree 8, 2×10^4 iterations are not enough, and along the flat directions very different coefficients give almost the same cost.

We also compared OLS, Ridge and Lasso fitted with the same plain gradient descent on the single test split (`test_mse.png` in our repository). These results mix the effect of the penalty with the implicit regularisation of unconverged gradient descent; at degree 15, for example, OLS gives a test MSE of 0.033 with gradient descent against 0.43 in closed form. We therefore compare the methods with converged solvers and cross-validation in Section III I. For the Lasso, the choice of optimiser matters: subgradient methods only approach the minimum to within an oscillation set by η and lose the sparsity, whereas ISTA or coordinate descent handle the penalty exactly.

H. Stochastic gradient descent

All results are for degree 5 after 500 epochs, so every run has evaluated the same number of single-point gradients. With a constant learning rate, the best η decreases with the mini-batch size, from $1/h_{\max}$ for full batch to $0.03/h_{\max}$ for $B = 1$, as expected since the largest single-point curvature, 84, is thirteen times $h_{\max}$. After 500 epochs, the relative excess cost for OLS is 2.0×10^{-3} for $B = 5$, 2.6–2.8×10^{-3} for $B = 1$ and 14, 7.8×10^{-3} for $B = 35$ and 3.1×10^{-3} for full batch. Per epoch, the small mini-batches do slightly better, because they make more updates. In wall time the picture is reversed: an epoch takes 0.65 ms with $B = 1$ and 0.06 ms with full batch, since with 70 points the cost of an update is dominated by the Python overhead. The advantage of SGD only appears when n is large enough to make full gradients expensive.

Figure 11 shows the effect of the learning-rate schedule for $B = 5$. A constant $\eta = 0.3/h_{\max}$ quickly reaches an excess cost of 3.0×10^{-2} but then stalls at its noise floor (1.7×10^{-2} after 500 epochs), while $\eta = 0.1/h_{\max}$ is slower but reaches 2.0×10^{-3}. The decaying schedule (32) with $\eta_0 = 1/h_{\max}$ removes this trade-off: it is fast initially and keeps improving, to 6.6×10^{-4}, three times better than the best constant rate and five times better than full batch. With $\eta_0 = 0.3/h_{\max}$ it decays too early (9.3×10^{-3}). Since the largest η_0 we tested was the best, a larger value might do better still. Ridge behaves the same way.

Table II. Relative excess training cost $(C - C^*)/C^*$ for OLS at degree 5 after 500 epochs, for each update rule with SGD ($B = 5$) with a constant and a decaying learning rate, and in full batch (500 iterations), each at its best learning rate. Medians over five shuffling seeds.

Method	SGD, constant	SGD, decaying	Full batch
GD	2.0×10^{-3}	6.6×10^{-4}	3.1×10^{-3}
Momentum	1.9×10^{-3}	7.2×10^{-4}	3.8×10^{-13}
AdaGrad	1.2×10^{-3}	3.7×10^{-2}	2.6×10^{-3}
RMSprop	2.6×10^{-3}	4.1×10^{-4}	1.0×10^{-3}
Adam	1.5×10^{-3}	4.3×10^{-4}	2.1×10^{-16}

Table II compares the update rules of Section II J. With stochastic gradients they become remarkably similar: all five end between 1.2×10^{-3} and 2.6×10^{-3} with a constant learning rate, and four of them between 4.1×10^{-4} and 7.2×10^{-4} with the decaying schedule. In full batch, the same 500 iterations take momentum and Adam to round-off level. The error is now limited by the gradient noise rather than by the conditioning, and momentum cannot remove noise. AdaGrad with the decaying schedule is the exception (3.7×10^{-2}): its built-in decay combined with the explicit one stops the iteration too early, while with a constant rate it is the best stochastic method. As before, the less converged runs often have a lower test MSE (0.037 to 0.050, against 0.0488 in closed form), an effect of implicit regularisation rather than of the method. For a small, convex problem like ours, SGD offers no advantage over full-batch momentum or Adam; its merits lie in large data sets and non-convex problems.

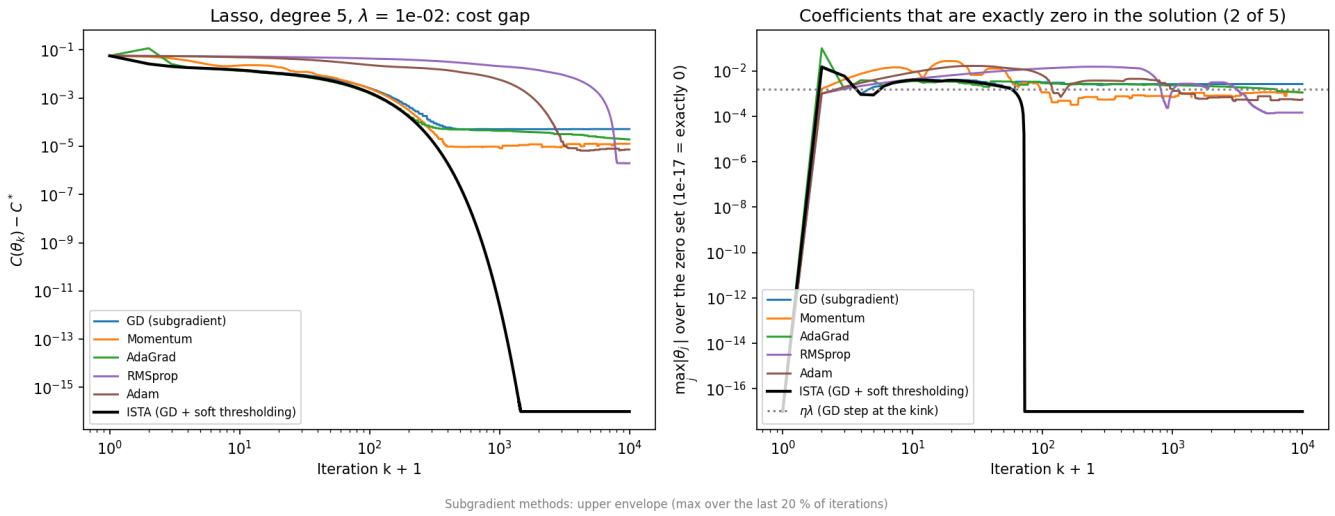


Figure 9. Lasso at polynomial degree 5 with $\lambda = 10^{-2}$, solved with the subgradient (33) and the methods of Sections II H and II J, and with ISTA (35) (black), for 10^4 iterations from $\boldsymbol{\theta}_0 = \mathbf{0}$. Left: cost relative to the minimum. Right: largest magnitude among the two coefficients that are exactly zero in the solution (10^{-17} means exactly zero); the dotted line marks $\eta\lambda$ for plain gradient descent. For the subgradient methods, the maximum over the last 20 % of the iterations is plotted. $n = 100$, $\sigma = 0.1$, 70 training points, seed 2026.

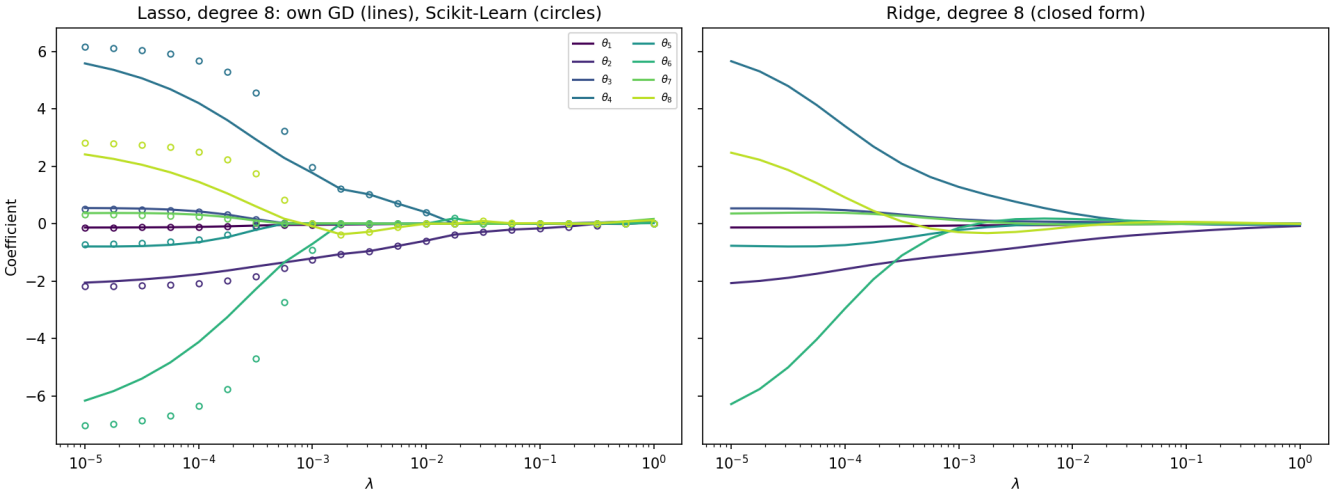


Figure 10. Coefficient paths as functions of λ at polynomial degree 8. Left: Lasso with our subgradient gradient descent ($\eta = 1/h_{\max}$, 2×10^4 iterations per λ, warm-started from the solution at the next larger λ) (lines) and Scikit-Learn with $\alpha = \lambda/2$ (circles). Right: Ridge (closed form) for the same values of λ. Coefficients refer to the standardised features. Same data as Fig. 9.

I. Final model selection

Of the 225 Lasso combinations of degree and λ, 64 (all at degree 8 or higher with $\lambda \leq 10^{-3}$) were excluded because they did not converge. Ridge at degree 8 with $\lambda = 10^{-4}$ has the lowest cross-validated error, 0.0287 ± 0.0027 (Fig. 12, Table III). The best Lasso model (degree 10, $\lambda = 10^{-3}$, 0.0297 ± 0.0029) lies within one standard error, so the two cannot be distinguished, whereas OLS (degree 6, 0.0346 ± 0.0031) is clearly worse. By our criterion the selected model is Ridge, with the Lasso as an equivalent alternative. The one-SE rule changes only the Lasso choice, to degree 6 with five of six coefficients non-zero, a simpler and sparser model. The selected OLS degree is lower than in Section III D because each training fold here contains only 56 points, in line with the dependence on n found in Section III C.

With a single 5- or 10-fold split and a different seed, OLS still selects degree 6 and the Lasso degree 10, but Ridge selects degree 6 ($\lambda = 3.2 \times 10^{-4}$) or degree 11 ($\lambda = 10^{-6}$). The error surface of Ridge is flat, since a

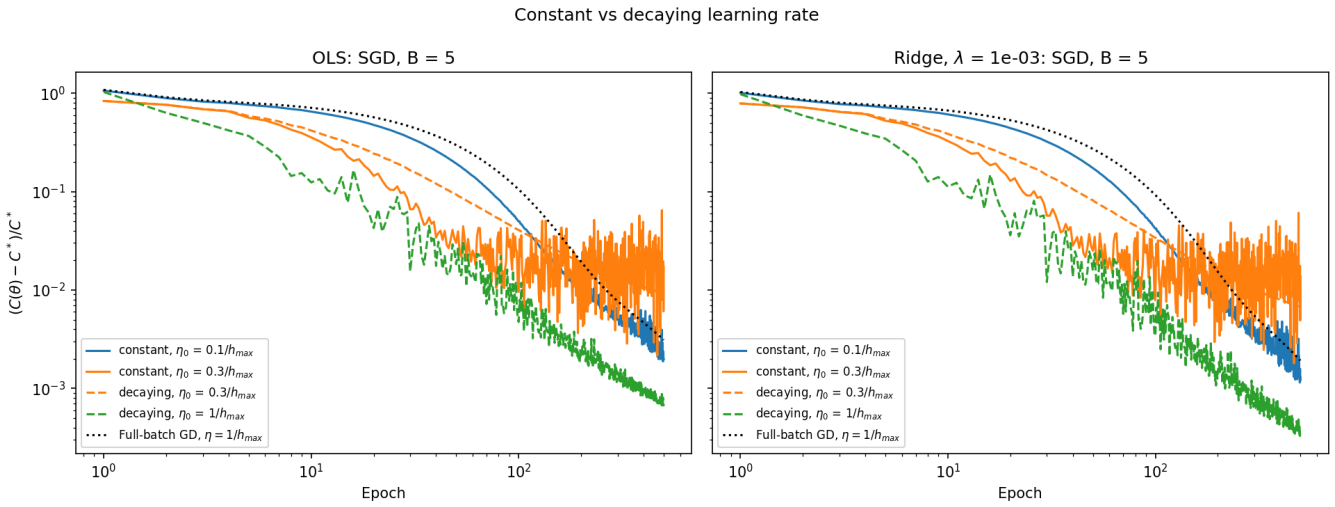


Figure 11. Plain SGD with $B = 5$ for OLS (left) and Ridge with $\lambda = 10^{-3}$ (right) at degree 5: constant learning rates $0.1/h_{\max}$ and $0.3/h_{\max}$ (solid) and the decaying schedule (32) with $\eta_0 = 0.3/h_{\max}$ and $1/h_{\max}$ (dashed), compared with full-batch gradient descent with $\eta = 1/h_{\max}$ (dotted). Relative excess training cost as a function of the epoch; medians over five shuffling seeds.

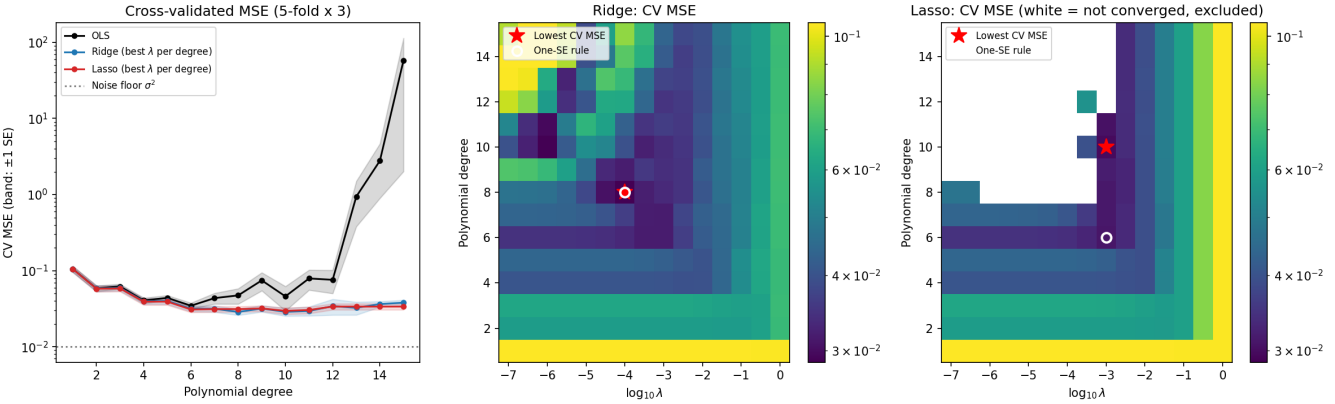


Figure 12. Cross-validated MSE on the 70 training points (5-fold cross-validation repeated three times, 15 folds; scaler fitted on the training folds). Left: $\mathrm{MSE}_{\mathrm{CV}}$ as a function of the polynomial degree for OLS and for Ridge and Lasso at the best λ for each degree; the bands show ± 1 standard error and the dotted line the noise level $\sigma^2 = 10^{-2}$. Middle and right: $\mathrm{MSE}_{\mathrm{CV}}$ over degree and λ for Ridge and Lasso (logarithmic colour scale, capped at the 95th percentile). The star marks the lowest $\mathrm{MSE}_{\mathrm{CV}}$, the circle the model chosen by the one-standard-error rule. White Lasso cells did not converge and are excluded. $n = 100$, $\sigma = 0.1$, seed 2026.

higher degree can be compensated by a larger penalty, so its specific degree and λ should not be over-interpreted; this justifies the repeated cross-validation. Evaluated once on the 30 test points, the selected models give test MSEs of 0.0158 (Ridge), 0.0179 (Lasso) and 0.0187 (OLS), the same ranking as cross-validation; the OLS and Ridge values coincide with those of the same models in Sections III A and III B. The test errors lie below the cross-validated ones, partly because the final models are trained on 70 rather than 56 points, and their differences of less than 20 % are not significant with 30 test points. The best test MSE is 60 % above σ^2.

Figure 13 shows which part of the error each method attacks. The absolute numbers are large, because the bootstrap includes the exterior test points and resamples that lack the edge points (Section III D), but this affects all methods equally. OLS attacks neither term and relies entirely on the choice of degree. At degree 12 its squared bias is 50 and its variance 3.1×10^3, and its variance grows from 0.17 at degree 5 to 40 at degree 10 and 5.8×10^5 at degree 15. Ridge attacks the variance: at degree 12, increasing λ from 10^{-7} to 1 reduces the variance from 90 to 2×10^{-3}, while the squared bias grows from 0.019 to 0.048, with the lowest error with respect to f, 0.042, at $\lambda = 0.32$. The Lasso also attacks the variance, but at a smaller cost in bias. Its variance falls from 0.76 to

Table III. Models selected by cross-validation on the training set (lowest $\mathrm{MSE_{CV}}$ and one-standard-error rule), with $\mathrm{MSE_{CV}}$ $\pm$ its standard error over 15 folds, and the test MSE, test R^2 and number of non-zero coefficients of the model refitted on the whole training set. The test set was used only for this table.

Method	Rule	Degree	λ	$\mathrm{MSE_{CV}}$	Test MSE	Test R^2	Non-zero
OLS	lowest CV / one-SE	6	–	0.0346 ± 0.0031	0.0187	0.823	6/6
Ridge	lowest CV / one-SE	8	10^{-4}	0.0287 ± 0.0027	0.0158	0.851	8/8
Lasso	lowest CV	10	10^{-3}	0.0297 ± 0.0029	0.0179	0.831	7/10
Lasso	one-SE	6	10^{-3}	0.0309 ± 0.0026	0.0190	0.821	5/6

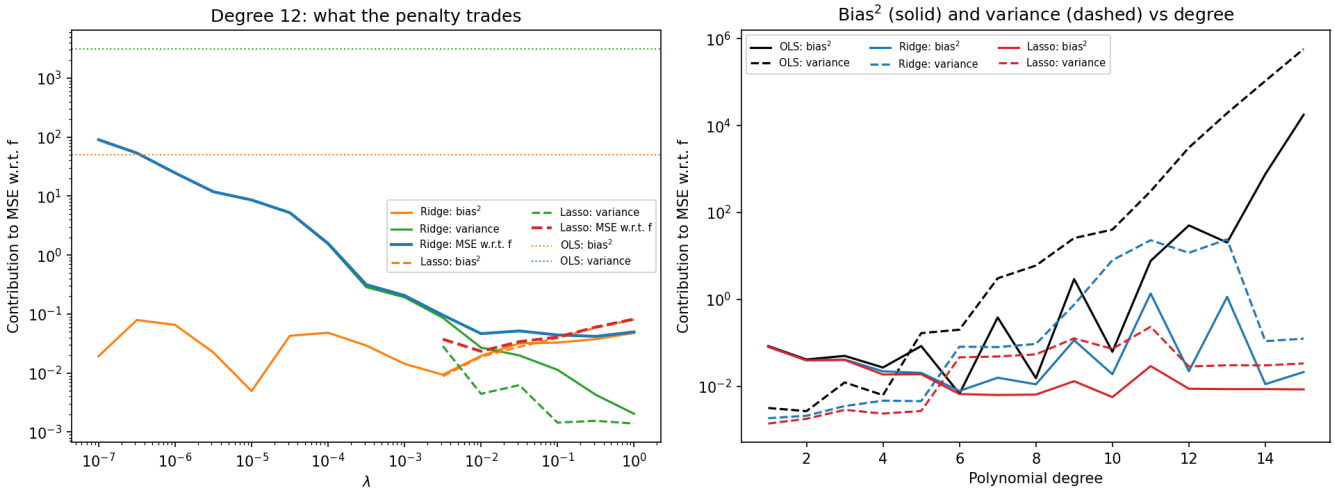


Figure 13. Bootstrap bias-variance decomposition with the true function $f(x_{\mathrm{test}})$ as reference (200 resamples of the training set, the same for every curve; 30 fixed test points). Left: squared bias, variance and error with respect to f for Ridge (solid) and Lasso (dashed) at degree 12 as functions of λ, with the OLS values at the same degree (dotted). Lasso is shown only where all cross-validation fits converged. Right: squared bias (solid) and variance (dashed) as functions of the degree for OLS, and for Ridge and Lasso at their cross-validated optimal λ for each degree.

1.4×10^{-3} and its squared bias rises from 4.7×10^{-3} to 0.081 as λ goes from 3×10^{-4} to 1, with the lowest error, 0.023, at $\lambda = 10^{-2}$, almost half the best Ridge value. At the cross-validated λ for each degree, the Lasso has the lowest variance of the three methods (at most 0.073 at degrees 5, 10 and 15) and, from degree 10, the lowest squared bias (5.6×10^{-3}, against 0.019 for Ridge). By setting some of the correlated monomials exactly to zero, it removes the near-degenerate directions instead of only damping them. The evidence thus supports the standard reading, that both penalties trade a moderate increase in bias for a large reduction in variance, with the nuance that the Lasso pays less in bias at high degree. The values of λ that minimise the bootstrap error are much larger than the cross-validated ones, since the bootstrap penalises variance more heavily.

OLS is the simplest method, with a closed form and nothing to tune, but it is very sensitive to the degree, and its variance explodes near the ends of the interval. Ridge also has a closed form, makes the result much less sensitive to the degree, improves the conditioning for gradient methods (Section III E) and gave the lowest cross-validated error. Its weaknesses are an extra parameter, a flat error surface that makes the chosen degree and λ unstable, and no sparsity. The Lasso gives sparse models, controlled the variance best in the bootstrap and was equivalent to Ridge in cross-validation, but it needs an optimiser that handles the non-smooth penalty (Section III G), did not converge for small λ at high degree, and its sparsity pattern among correlated monomials is not stable. We would choose Ridge or the Lasso over OLS. More important than the choice between them is that all cross-validated errors remain about three times the noise level: a global polynomial in the monomial basis is a poor representation of the Runge function, whichever method is used to fit it.

IV. CONCLUSION

We have fitted polynomials to noisy data from the Runge function $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ with OLS, Ridge and Lasso regression, using both closed-form solutions and our own gradient descent codes, and we used the bootstrap and cross-validation to assess and select models. For OLS, a single train/test split gave an erratic test error whose minimum at degree 14 turned out to be an artefact of that split. The bootstrap and

cross-validation instead showed that the error is lowest at intermediate degrees as long as extrapolation to the ends of the interval is excluded, and that the optimal degree grows with the number of data points, in agreement with Hastie et al. [3]; extrapolation near $x = \pm 1$ was the main source of variance throughout. In the final cross-validated comparison on the training set, Ridge at degree 8 with $\lambda = 10^{-4}$ had the lowest error, $\mathrm{MSE}_{\mathrm{CV}} = 0.0287 \pm 0.0027$, with the Lasso (degree 10, $\lambda = 10^{-3}$) within one standard error and OLS (degree 6) clearly worse; on the held-out test set the three models gave test MSEs of 0.0158, 0.0179 and 0.0187. The bootstrap decomposition confirmed that both penalties reduce the variance by several orders of magnitude at the cost of a moderate increase in bias, with the Lasso doing so at the smallest cost in bias for high-degree models.

The main obstacle for the iterative methods was the conditioning of the problem. The strongly correlated polynomial columns give the Hessian a condition number of $\kappa \approx 3 \times 10^6$ at degree 10, which makes plain gradient descent impractically slow. At degree 5, momentum reached a relative parameter error of 10^{-6} after 256 iterations (194 with the optimal heavy-ball parameters), against about 5700 for plain gradient descent and AdaGrad, while RMSprop and Adam did not converge to this precision with a constant learning rate. Mini-batch SGD did not improve on this: with a constant learning rate it stalled on a noise floor at a relative excess cost of about 10^{-3}, a decaying learning rate improved this by a factor of three to five, and the large advantage of momentum and Adam in full batch disappeared with stochastic gradients. For our small data set, full-batch methods were also faster in wall time. The ranking of the optimisers depended strongly on the problem and on the learning rate, so no single method was best overall.

For a problem of this size, the closed-form solutions give the best balance between computational cost and quality of fit, provided they are computed with the SVD or the pseudoinverse rather than by inverting the ill-conditioned matrix $\boldsymbol{X}^T\boldsymbol{X}$. Ridge regression with a cross-validated λ gives a robust fit at low cost. Iterative methods are only needed when no closed-form solution exists, as for Lasso, or when the number of parameters is large.

Our study is limited to one-dimensional synthetic data, mostly a single random seed, and learning rates that were not tuned systematically for each optimiser, so the ranking of the methods is only indicative. Most importantly, a global monomial basis is poorly suited to the Runge function, and this is the root cause of the ill-conditioning. Future work should use orthogonal polynomials such as Legendre or Chebyshev polynomials, attach bootstrap error bars to the selected degree and λ, and compare the optimisers with individually tuned learning rates.

APPENDIX A: USE OF AI/LLM TOOLS

A.1 Tools used. Claude (Claude Opus 5.5, accessed via claude.ai, September–October 2026)

Summary.

A.2 Text writing and editing.

- 0: No AI was used writing this
- 1: Language only
- 2: Editorial. We asked AI to rewrite our own sentences.
- 3: Generated. We gave the LLM a prompt and it generated a text. We then revised it to make sure it was correct.

Section	Level	Notes
Abstract	0	Written by us
Introduction	0	Written by us
Method	3	Mostly written by us; Claude wrote some sections with revisions by us
Results	3	Written by Claude with revision by us
Discussion	3	Written by Claude with revision by us
Conclusion	3	Written by Claude with revision by us

A.3 Code generation and assistance. The levels given here are

- 0: No AI was used.
- 1: We wrote the script and only used AI for debugging
- 2: Most of the script was written by us, but AI helped to write some snippets.
- 3: The skeleton of the code was written by AI, and we wrote on top of that.
- 4: A substantial amount of AI was used to write the code.

File	Level	Description
`utils.py`	4	Shared functions: data, scaling, regression, resampling, gradients and optimisers.
`Project1aa.py`	4	OLS vs. degree.
`Project1bb.py`	4	Ridge vs. degree and λ.
`Project1cc.py`	4	Bootstrap bias-variance analysis.
`Project1dd.py`	4	k-fold cross-validation.
`Project1ee.py`	4	Gradient descent, analytical and JAX gradients.
`Project1ff.py`	4	Momentum and adaptive learning rates.
`Project1gg.py`	4	Lasso with subgradient methods and ISTA.
`Project1hh.py`	4	Stochastic gradient descent.
`Project1ii.py`	4	Final model selection.

All functions carry an `LLM-assisted` tag in their docstring.

`Project1aa` to `ee.py` were based on code written by us, but heavily altered afterwards. `Project1ff`-to `ii.py` was written by Claude with our guidance.

APPENDIX B: SUPPLEMENTARY FIGURES

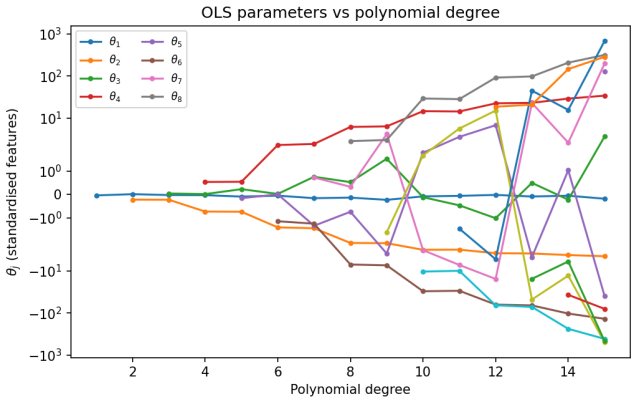


Figure 14. OLS coefficients θ_j for the standardised features as functions of polynomial degree, on a symmetric logarithmic axis. Only θ_1–θ_8 are labelled. Same settings as Fig. 1.

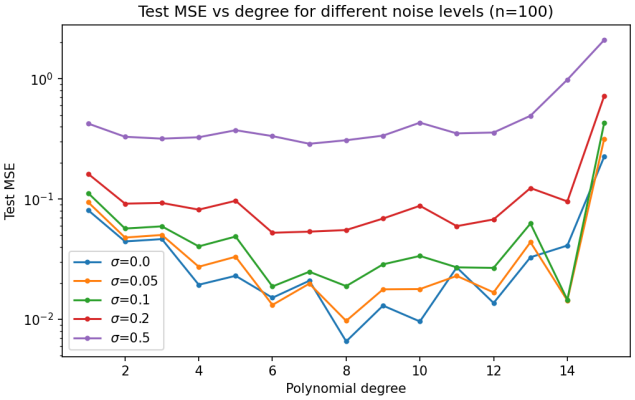


Figure 15. Test MSE for OLS as a function of polynomial degree for noise levels $\sigma = 0, 0.05, 0.1, 0.2$ and 0.5, with $n = 100$, a 70/30 train/test split and seed 2026. All curves use the same x values and the same standard normal draws, scaled by σ, so they differ only in the noise level.

[1] M. Hjort-Jensen, *Machine Learning and the Physical Sciences* (Department of Physics, University of Oslo, Norway, 2026) p. xvi.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research **12**, 2825 (2011).

[3] T. Hastie, R.Tibshirani, and J.Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition. Springer Series in Statistics* (Springer, New York, 2009) pp. 389–414.

[4] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs, `http://github.com/jax-ml/jax` (2018).

[5] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, Automatic differentiation in machine learning: a survey, Journal of Machine Learning Research **18**, 1 (2018).

[6] B. T. Polyak, Some methods of speeding up the convergence of iteration methods, USSR Computational Mathematics and Mathematical Physics **4**, 1 (1964).

[7] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in *3rd International Conference on Learning Representations (ICLR)* (2015) arXiv:1412.6980.

[8] A. Beck and M. Teboulle, A fast iterative shrinkage-thresholding algorithm for linear inverse problems, SIAM Journal on Imaging Sciences **2**, 183 (2009).

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016) `http://www.deeplearningbook.org`.